Universidad de La Laguna

Escuela de Doctorado y Estudios de Posgrado

Máster Universitario en Ciberseguridad e Inteligencia de Datos

Estudio e implementación de modelos de lenguaje de gran tamaño (Large Language Models, LLM) para problemas de clasificación

Autor: Paul Santos Ramos

Tutor: Carlos Pérez González

La Laguna, junio de 2025

D. Carlos Pérez González, con N.I.F. 45.452.719-G, Profesor Titular de Universidad adscrito al Departamento de Matemáticas, Estadística e Investigación Operativa de la Universidad de La Laguna, como tutor,

CERTIFICA:

Que la presente memoria titulada:

"Estudio e implementación de modelos de lenguaje de gran tamaño (Large Language Models, LLM) para problemas de clasificación"

ha sido realizada bajo su dirección por D. Paul Santos Ramos, con N.I.F. 45.865.416-N.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firma la presente en La Laguna, a 11 de julio de 2025.

Resumen

This work explores the automatic classification of commercial products using Large Language Models (LLMs), a key task in domains such as e-commerce, retail analytics, and inventory management. Two primary approaches are compared: *fine-tuning*, which involves supervised adaptation of pre-trained models using labeled data, and *zero-shot inference*, where models classify without any task-specific retraining.

The study builds upon recent advances in Natural Language Processing (NLP), particularly following the emergence of the Transformer architecture (Vaswani et al., 2017), which replaced recurrent structures with attention mechanisms capable of modeling long-range dependencies efficiently. Models such as BERT, RoBERTa, and GPT have enabled significant progress in textual classification thanks to their capacity to encode contextual meaning.

The models are evaluated on real-world Spanish-language data provided by ISTAC, comprising short product descriptions annotated with domain-specific hierarchical labels and suffixes. The dataset exhibits class imbalance and lexical ambiguity, posing challenges for generalization and disambiguation. To address this, both compact and standard LLMs were used, including monolingual, multilingual, and distilled variants. All training was performed on local hardware using the Hugging Face `transformers` library and 4-fold cross-validation. In contrast, *zero-shot* evaluations were conducted using quantized models in `GGUF` format executed in LM Studio, a tool optimized for local CPU inference.

Results show that fine-tuned models such as `beto`, `distilbeto`, and `maria` consistently exceed 90 % accuracy and weighted F1-score, highlighting their suitability for deployment in real-world classification systems. Compact models like `mobilebert` also performed well, suggesting their applicability in constrained environments such as mobile or embedded devices. Conversely, *zero-shot* models exhibited high error rates, particularly on rare or semantically overlapping labels, although some—like `Qwen` or `Mistral`—showed potential when guided by prompt engineering.

Limitations include the absence of hyperparameter tuning, uniform training strategies across models, and computational constraints that limited training to a small number of epochs. Despite these constraints, the study demonstrates that robust classification systems can be developed with limited infrastructure, providing reproducible methods suitable for real-world, resource-constrained scenarios.

This work concludes that LLMs, especially when fine-tuned, are effective tools for text classification in specialized domains. Future directions include extending the study to other domains, exploring few-shot methods, balancing minority classes more actively, and improving semantic disambiguation for ambiguous or artificially labeled categories.

Índice general

Índice general **1**

1. Introducción **3**
1.1. Motivación y contexto del estudio . . . 3
1.2. Objetivos del trabajo . . . 3
1.3. Evolución histórica de los modelos neuronales . . . 4
1.4. Conceptos fundamentales . . . 5

2. Modelos de Lenguaje de Gran Tamaño (LLMs) **8**
2.1. Introducción . . . 8
2.2. Definición y funcionamiento de los LLMs . . . 8
2.3. Aplicación a tareas de clasificación . . . 9
2.4. Plataformas empleadas . . . 9
2.5. Modelos utilizados en este estudio . . . 11

3. Metodología de entrenamiento y evaluación de LLMs **13**
3.1. Introducción . . . 13
3.2. Conjuntos de Datos Utilizados para el Entrenamiento . . . 13
3.3. Ajuste fino (Fine-Tuning) de modelos preentrenados . . . 15
3.4. Aprendizaje sin ajuste (Zero-shot learning) . . . 18
3.5. Limitaciones computacionales y justificación del enfoque . . . 20

4. Análisis de los resultados del problema de clasificación **21**
4.1. Introducción . . . 21
4.2. Análisis por métricas de clasificación . . . 21
4.3. Análisis por grupo de modelos y arquitectura para modelos entrenados mediante *fine-tuning* . . . 30
4.4. Relación entre tamaño del modelo y rendimiento predictivo . . . 35
4.5. Análisis de errores tras ajuste fino: patrones, frecuencia y causas . . . 37
4.6. Discusión de los resultados obtenidos . . . 49

5. Discusión y Conclusiones **52**
5.1. Introducción . . . 52
5.2. Discusión de resultados . . . 52
5.3. Conclusiones generales . . . 52
5.4. Líneas futuras de investigación . . . 53

6. Discussion and Conclusions **54**
6.1. Introduction 54
6.2. Discussion of Results 54
6.3. General Conclusions 54
6.4. Future Research Directions 55

Bibliografía **56**

A. Apéndice **57**

Capítulo 1

Introducción

1.1. Motivación y contexto del estudio

La clasificación automática de productos es una tarea clave en entornos comerciales como supermercados o plataformas de comercio electrónico. Una categorización precisa facilita la gestión del inventario, mejora la experiencia de búsqueda del cliente y permite análisis más robustos de los patrones de consumo.

En los últimos años, los *Transformers* han revolucionado el procesamiento del lenguaje natural. Este tipo de arquitectura, introducida por Vaswani et al. (2017), reemplaza los mecanismos recurrentes por un esquema de atención que capta relaciones de largo alcance en el texto. Dentro de este marco surge **BERT** (Bidirectional Encoder Representations from Transformers), un modelo que aprende representaciones contextuales bidireccionales al preentrenarse sobre grandes corpus de texto. Su éxito ha inspirado numerosas variantes que adaptan el mismo principio a distintos tamaños de red, lenguajes y dominios.

La disponibilidad de estos modelos de lenguaje de gran tamaño (*Large Language Models*, LLMs) abre dos caminos complementarios para la clasificación textual:

1. **Ajuste supervisado (*fine-tuning*).** Se parte de un modelo preentrenado y se entrena con ejemplos etiquetados del dominio comercial para especializarlo en la tarea.

2. **Aprendizaje sin ajuste *zero-shot*.** Se emplea el modelo tal cual, formulando instrucciones en lenguaje natural sin reentrenamiento, lo que permite evaluar su conocimiento implícito.

1.2. Objetivos del trabajo

El objetivo principal de este Trabajo Fin de Máster es comparar el rendimiento de distintos LLMs en la clasificación de productos comerciales, analizando tanto el ajuste supervisado como el enfoque *zero-shot*. Para ello se diseña un experimento que:

- Evalúa el rendimiento global y por clases minoritarias en escenarios realistas con etiquetas desbalanceadas y descripciones ambiguas.

- Considera restricciones computacionales típicas de un entorno académico o empresarial con recursos limitados.

- Ofrece una visión integral del estado actual de los LLMs para la clasificación textual en el dominio comercial.

Los modelos concretos utilizados en cada enfoque se describen con detalle en la sección dedicada a la experimentación, a fin de mantener esta introducción concisa y centrada en el marco conceptual.

1.3. Evolución histórica de los modelos neuronales

El concepto de neurona artificial fue propuesto por primera vez en 1943 por McCulloch y Pitts como un modelo lógico básico. Sin embargo, no fue hasta los años 80, con la introducción del algoritmo de *backpropagation*, que las redes neuronales comenzaron a ser entrenadas de forma eficaz [1].

Posteriormente, las redes neuronales profundas (Deep Neural Networks) impulsaron avances notables en visión artificial y procesamiento del lenguaje [2]. En 2017, el artículo *"Attention is All You Need"* de Vaswani et al. revolucionó el campo introduciendo el modelo Transformer, que eliminaba la dependencia secuencial de las redes recurrentes, permitiendo entrenar modelos con mayor paralelismo y capacidad contextual [1].

Desde entonces, la comunidad ha desarrollado múltiples variantes y optimizaciones del Transformer, como BERT, RoBERTa, DeBERTa, GPT y sus derivados, los cuales han sido entrenados con corpus masivos y cientos de millones o miles de millones de parámetros, dando lugar a los actuales Modelos de Lenguaje de Gran Tamaño (LLMs) [1], [2], [3].

Evolución de los problemas de clasificación

La clasificación automática ha sido una preocupación central del aprendizaje automático desde sus orígenes, con aplicaciones que abarcan desde la medicina y la visión por computador hasta el análisis de texto y la categorización de productos. En las primeras décadas, los problemas de clasificación se afrontaron mediante **algoritmos lineales** —por ejemplo, regresión logística y máquinas de vectores soporte (SVM)—, idóneos para conjuntos de datos estructurados y de baja dimensionalidad. Su eficacia dependía de una rigurosa *ingeniería de características*: la selección o construcción manual de variables explicativas relevantes. Este requisito limitaba la escalabilidad del enfoque y dificultaba capturar relaciones no lineales.

El advenimiento de las **redes neuronales multicapa** (MLP) permitió modelar dependencias más complejas. Sin embargo, estos modelos adolecían de problemas de entrenamiento (desvanecimiento del gradiente, sobreajuste) y exigían grandes volúmenes de datos anotados.

La irrupción del **aprendizaje profundo** (*deep learning*) cambió el panorama: arquitecturas convolucionales y, más tarde, la arquitectura **Transformer** han incrementado de forma drástica la capacidad para aprender representaciones jerárquicas desde datos brutos, especialmente en procesamiento del lenguaje natural (PLN). Los *Large Language Models* (LLMs), preentrenados sobre corpus masivos, permiten hoy abordar la clasificación directamente sobre texto crudo, suprimiendo en gran medida la necesidad de ingeniería de variables y habilitando esquemas como el *fine-tuning* y el *zero-shot learning*. Esta evolución ha derivado en sistemas más precisos, escalables y adaptables, capaces de lidiar con clases desbalanceadas, descripciones ambiguas y taxonomías muy específicas.

Arquitectura Transformer y ecosistema de modelos LLM

La arquitectura **Transformer**, presentada en 2017 por Vaswani et al., representa uno de los avances más significativos en el procesamiento automático del lenguaje natural. A diferencia de modelos anteriores como las redes neuronales recurrentes, los Transformers pueden analizar una secuencia completa de texto de manera simultánea, lo que les permite comprender mejor el contexto general de una oración y procesar grandes volúmenes de datos de forma mucho más eficiente.

Su funcionamiento se basa en un mecanismo denominado *autoatención*, que permite al modelo identificar qué palabras son más relevantes en cada momento en función del contexto. Esto le otorga una gran capacidad para resolver ambigüedades y captar relaciones entre elementos distantes del texto. Por ejemplo, puede distinguir si un pronombre como "él" se refiere al sujeto o al objeto de una oración, sin necesidad de seguir un orden palabra por palabra.

La estructura de un Transformer incluye varios componentes esenciales: un módulo de autoatención que selecciona las partes más importantes del texto, una red neuronal interna que transforma esa información, mecanismos de normalización que estabilizan el aprendizaje, y codificaciones posicionales que indican la ubicación de cada palabra en la secuencia. Gracias a este diseño modular, los Transformers han podido escalarse a tamaños masivos y dar lugar a los llamados Modelos de Lenguaje de Gran Tamaño (LLMs), como BERT, GPT, RoBERTa o LLaMA, capaces de generar texto, responder preguntas, traducir, resumir o clasificar documentos, entre muchas otras tareas.

Estos modelos están disponibles a través de distintos canales. Algunos, como los ofrecidos por OpenAI, Anthropic o Google, funcionan mediante interfaces web o APIs, es decir, el usuario envía una consulta textual y recibe una respuesta sin necesidad de instalar nada. Otros, como los desarrollados por Meta, Microsoft o Mistral, pueden descargarse y ejecutarse localmente, lo que permite mayor control sobre la privacidad y los recursos. Además, existen comunidades como Hugging Face, BigScience o EleutherAI que publican modelos completamente abiertos, facilitando su uso, adaptación y distribución. Herramientas como LM Studio, llama.cpp o Ollama permiten ejecutar estos modelos en ordenadores personales sin conexión a internet. La elección entre uno u otro formato depende de factores como la disponibilidad de hardware, la sensibilidad de los datos o los objetivos del proyecto. En este trabajo se han empleado tanto modelos locales como servicios en la nube, aprovechando las ventajas de ambos enfoques para realizar una evaluación comparativa.

1.4. Conceptos fundamentales

Para comprender el funcionamiento interno de los Modelos de Lenguaje de Gran Tamaño (LLMs), es útil introducir algunos conceptos técnicos esenciales. Aunque estos modelos se basan en arquitecturas complejas, todos ellos comparten una serie de componentes básicos que permiten procesar texto, aprender patrones y generar respuestas coherentes. A continuación se explican los elementos más relevantes de forma accesible:

- **Neurona artificial**: Es la unidad computacional más simple de una red neuronal. Inspirada en las neuronas biológicas, su funcionamiento consiste en recibir un conjunto de entradas numéricas, multiplicarlas por ciertos valores llamados *pesos*, sumar los productos y añadir un valor denominado *sesgo*. Finalmente, aplica una función de activación que transforma ese resultado en una salida. Esta operación se expresa comúnmente como:

  $$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

  donde x_i son las entradas, w_i los pesos, b el sesgo y f la función de activación. Millones de estas neuronas se interconectan para formar redes complejas.

- **Capas ocultas y arquitectura de red**: Las redes neuronales están compuestas por múltiples capas de neuronas organizadas secuencialmente. Las capas de entrada reciben los datos iniciales (por ejemplo, una secuencia de tokens), y las capas de salida entregan el resultado final (como una clasificación o una predicción de texto). Entre ambas se sitúan las llamadas *capas ocultas*, donde se llevan a cabo las transformaciones intermedias más importantes. En cada una de estas capas, el modelo aprende a extraer representaciones progresivamente más

abstractas de los datos, permitiéndole capturar patrones complejos del lenguaje. Cuanto mayor es el número de capas ocultas, mayor es la profundidad del modelo y su capacidad para aprender relaciones de alto nivel, aunque también se incrementa el coste computacional.

- **Pesos de la red**: Los pesos son los parámetros ajustables que definen la fortaleza de las conexiones entre neuronas. Durante el entrenamiento, estos valores se ajustan mediante algoritmos de optimización para reducir el error cometido en una tarea específica. En los LLMs, el número total de pesos puede superar los mil millones, lo que dota al modelo de una notable capacidad de generalización y aprendizaje del lenguaje.
- **Tokens y tokenización**: Los modelos de lenguaje no interpretan directamente letras o palabras, sino unidades mínimas llamadas *tokens*. Un token puede ser una palabra, una subpalabra o un carácter individual, según el método de tokenización utilizado. Por ejemplo, la palabra "supermercado" podría dividirse en los tokens `super`, `merca`, `do`. Esta fragmentación permite una mejor gestión de palabras poco frecuentes o desconocidas. Métodos comunes incluyen WordPiece, Byte-Pair Encoding (BPE) o SentencePiece.
- **Embeddings**: Cada token se convierte en un vector numérico denso conocido como *embedding*, que captura propiedades semánticas y sintácticas del término representado. Esto permite al modelo operar en un espacio continuo donde palabras con significados similares están representadas por vectores cercanos. Por ejemplo, los términos "rey" y "reina" o "doctor" y "médico" se ubicarán en regiones próximas del espacio de embedding. Estos vectores se aprenden automáticamente durante el entrenamiento.
- **Cabezas de atención (attention heads)**: Dentro de cada capa del Transformer, se emplean varias cabezas de atención paralelas. Cada una se especializa en detectar distintos tipos de relaciones entre tokens, lo que mejora la comprensión contextual del modelo. Este mecanismo es esencial para que los LLMs capten simultáneamente aspectos gramaticales, semánticos y estructurales.

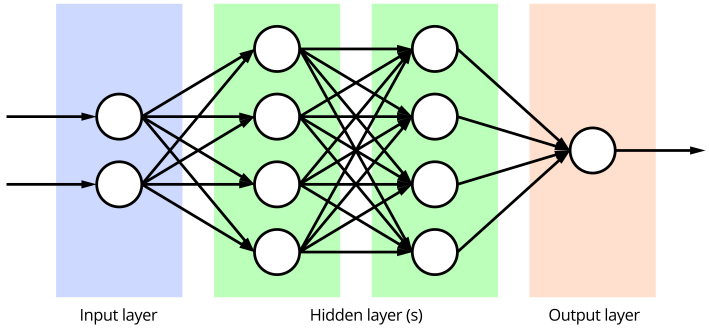

Figura 1: Ejemplo esquemático de una red neuronal multicapa, con capa de entrada, capas ocultas intermedias y capa de salida. Cada círculo representa una neurona artificial conectada mediante pesos ajustables.

Aplicaciones actuales de los modelos de lenguaje

Los Modelos de Lenguaje de Gran Tamaño (LLMs) no solo han supuesto una revolución a nivel teórico en el campo del aprendizaje profundo, sino que han alcanzado una notable madurez

tecnológica, siendo aplicados en una amplia variedad de contextos reales. Gracias a la arquitectura Transformer y al uso de grandes corpus de texto durante el entrenamiento, estos modelos son capaces de comprender, generar y manipular lenguaje natural con un alto grado de precisión.

En la práctica, los LLMs se utilizan en tareas como la generación automática de texto (redacción de correos, artículos, código, etc.), la traducción automática entre lenguas, la búsqueda semántica (donde se prioriza el significado por encima de las coincidencias exactas de palabras), la respuesta a preguntas en lenguaje natural, el análisis de sentimientos, la detección de contenido inadecuado o la clasificación de textos según diferentes criterios. También son fundamentales en sistemas conversacionales avanzados, asistentes virtuales y motores de recomendación.

Una de las características más relevantes de estos modelos es su capacidad para generalizar a nuevos dominios y tareas con una intervención mínima por parte del usuario. A través de técnicas como el *zero-shot learning*, es posible aplicar un modelo preentrenado a tareas no vistas previamente sin necesidad de entrenamiento adicional. Alternativamente, mediante el *fine-tuning*, se puede adaptar el modelo a tareas específicas utilizando un volumen reducido de datos etiquetados.

Esta versatilidad, combinada con su escalabilidad y eficiencia en la comprensión del contexto lingüístico, convierte a los LLMs en herramientas estratégicas en ámbitos donde el lenguaje es un componente central. En el presente trabajo se explora precisamente esta capacidad, evaluando distintas estrategias —con y sin ajuste fino— para abordar la tarea de clasificación automática de productos a partir de su descripción textual, en el contexto del comercio electrónico.

Capítulo 2

Modelos de Lenguaje de Gran Tamaño (LLMs)

2.1. Introducción

Los Modelos de Lenguaje de Gran Tamaño (LLMs) representan uno de los avances más significativos en el campo del procesamiento del lenguaje natural en la última década. Gracias a su capacidad para capturar patrones complejos en grandes volúmenes de texto, estos modelos permiten abordar tareas lingüísticas con un alto grado de precisión, desde la generación automática hasta la clasificación semántica.

Este capítulo presenta los principios fundamentales de los LLMs, su funcionamiento interno y su aplicación a tareas de clasificación textual. Se describen las principales arquitecturas utilizadas en este estudio, incluyendo tanto modelos entrenados mediante ajuste fino como modelos aplicados en modo *zero-shot*. Asimismo, se detallan las características de cada modelo empleado, su origen, tamaño y condiciones de ejecución, proporcionando así una base sólida para interpretar los resultados experimentales presentados en los capítulos posteriores.

2.2. Definición y funcionamiento de los LLMs

Los Modelos de Lenguaje de Gran Tamaño, o *Large Language Models* (LLMs), son redes neuronales entrenadas sobre grandes corpus de texto con el objetivo de modelar el lenguaje natural. Están diseñados para predecir la probabilidad de una secuencia de palabras, lo que les permite realizar tareas de comprensión, generación y clasificación textual.

Los LLMs modernos están basados en la arquitectura Transformer, propuesta por Vaswani et al. (2017), que introdujo el mecanismo de atención como pilar central. A diferencia de las redes recurrentes, los Transformers permiten capturar dependencias a largo plazo de manera eficiente y paralela.

Un modelo Transformer se compone de varias capas de atención y proyecciones lineales, donde cada token de entrada es transformado a través de múltiples etapas. Las entradas son primero convertidas en **embeddings** y enriquecidas con información posicional, y luego son procesadas por capas sucesivas que capturan las relaciones contextuales entre los tokens. Cada capa ajusta sus **pesos** internos mediante retropropagación y optimización de una función de pérdida, como la entropía cruzada en tareas de clasificación.

2.3. Aplicación a tareas de clasificación

Los Modelos de Lenguaje de Gran Tamaño (LLMs) pueden adaptarse con facilidad a tareas de clasificación de texto, gracias a su capacidad para generar representaciones contextuales profundas. A partir de una entrada textual, estos modelos producen un vector que resume su contenido —habitualmente asociado al token especial `[CLS]` o al último estado oculto—, el cual puede emplearse como base para predecir una etiqueta perteneciente a un conjunto predefinido.

Este tipo de arquitectura permite aplicar múltiples estrategias de clasificación, desde aquellas que incorporan capas adicionales entrenables hasta otras que formulan la tarea como una inferencia directa. De este modo, los LLMs ofrecen un marco flexible que puede adaptarse a distintos escenarios y necesidades, manteniendo una integración coherente entre representación lingüística y decisión categórica.

2.4. Plataformas empleadas

En el presente estudio se han utilizado dos enfoques complementarios para el uso de modelos de lenguaje de gran tamaño: por un lado, el entrenamiento y evaluación de modelos mediante la biblioteca `transformers` de la plataforma Hugging Face, y por otro, la ejecución local de modelos cuantizados mediante la herramienta `LM Studio`. Ambas estrategias ofrecen ventajas diferenciadas en términos de flexibilidad, eficiencia y privacidad, lo que permite comparar sus rendimientos y valorar su aplicabilidad en distintos contextos.

Hugging Face: ecosistema modular para el procesamiento del lenguaje

Hugging Face se ha consolidado como una de las plataformas más influyentes en el ámbito del aprendizaje automático aplicado al lenguaje natural [4]. Su propuesta se basa en un ecosistema modular que permite abordar todas las etapas del ciclo de vida de un modelo de lenguaje, desde la tokenización hasta la inferencia optimizada.

El componente más conocido de Hugging Face es el *Model Hub*, un repositorio colaborativo donde se pueden encontrar miles de modelos preentrenados listos para ser reutilizados o adaptados a tareas específicas. Su funcionamiento es análogo al de GitHub, pero orientado al intercambio de modelos y recursos para NLP.

Como se muestra en la Figura 1, el ecosistema de Hugging Face está organizado en torno a varios módulos clave:

- **Transformers**: Biblioteca central para cargar y utilizar modelos basados en la arquitectura Transformer. Incluye soporte para tareas como clasificación, traducción, generación de texto o respuesta a preguntas.
- **Tokenizers**: Encargada de convertir texto en secuencias de tokens comprensibles para los modelos, optimizando velocidad y compatibilidad con distintos formatos.
- **Datasets**: Proporciona acceso estandarizado a cientos de conjuntos de datos para entrenamiento y evaluación, con funciones eficientes de carga y preprocesamiento.
- **Accelerate**: Permite escalar el entrenamiento e inferencia de modelos en entornos con múltiples GPUs o TPUs, simplificando la configuración para usuarios avanzados.

Esta estructura modular, combinada con una interfaz sencilla y una comunidad activa, ha convertido a Hugging Face en la herramienta de referencia para el desarrollo ágil y reproducible de soluciones de NLP, tanto en investigación como en producción.

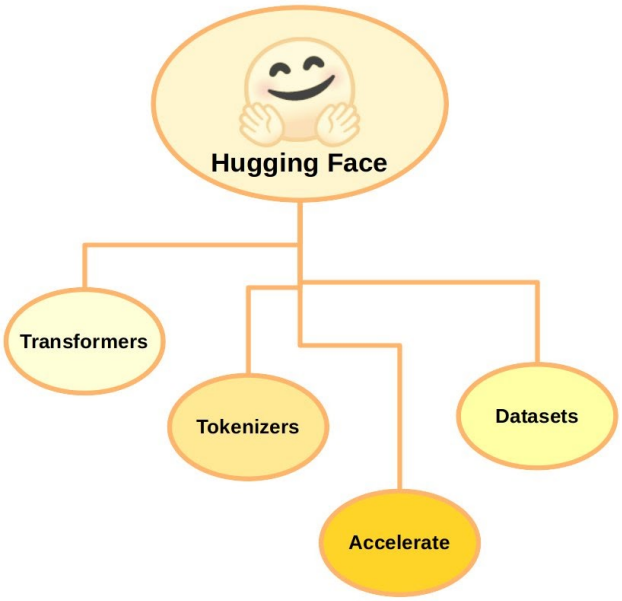

Figura 1: Componentes principales del ecosistema de Hugging Face: un entorno modular que conecta modelos, tokenizadores, conjuntos de datos y herramientas de optimización.

LM Studio y el formato `GGUF`

LM Studio es una aplicación de escritorio multiplataforma que facilita la ejecución de modelos de lenguaje de forma completamente local, sin necesidad de conexión a servidores externos o uso de APIs comerciales [5]. Esta herramienta está construida sobre el motor de inferencia `llama.cpp`, una implementación optimizada en C++ que permite ejecutar modelos de lenguaje cuantizados con gran eficiencia en CPU y GPU convencionales.

El formato utilizado por LM Studio es `GGUF` (GPT-Generated Unified Format), un contenedor binario que incluye los pesos del modelo —cuantizados para reducir el tamaño y acelerar la inferencia— así como la información necesaria para su configuración (tokenizadores, vocabularios, instrucciones de uso, etc.). Este formato ha ganado popularidad por su portabilidad, bajo consumo de recursos y compatibilidad con múltiples variantes de modelos.

Entre las principales características de LM Studio destacan:

- Una interfaz gráfica intuitiva que permite seleccionar modelos, redactar entradas (prompts) y visualizar respuestas de manera rápida y cómoda.
- Compatibilidad con distintos niveles de cuantización, lo que facilita la adaptación del modelo al hardware disponible sin comprometer en exceso el rendimiento.
- Posibilidad de trabajar completamente fuera de línea, una ventaja significativa en términos de privacidad, confidencialidad de los datos y autonomía del sistema.

En este trabajo, LM Studio se ha utilizado para ejecutar modelos de lenguaje en modo *zero-shot*, es decir, sin ningún ajuste posterior a su entrenamiento original. Esto permite evaluar su capacidad general de comprensión y clasificación de texto directamente a partir de instrucciones en lenguaje natural, y compararla con los modelos afinados a través de Hugging Face.

La combinación de ambas plataformas —una orientada al ajuste personalizado y otra a la inferencia local sin dependencias externas— proporciona un marco robusto y versátil para el análisis experimental de distintas aproximaciones en tareas de clasificación automática.

2.5. Modelos utilizados en este estudio

A lo largo del presente trabajo se ha empleado una variedad de modelos de lenguaje (LLMs) con diferentes tamaños, arquitecturas y configuraciones, tanto en inglés como en español. Todos los modelos fueron ejecutados localmente mediante el framework LM Studio en formato `GGUF`.

Modelos de Hugging Face

Los modelos utilizados en este estudio fueron seleccionados por su popularidad, disponibilidad y diversidad arquitectónica, abarcando distintas configuraciones de tamaño y capacidad. Con el objetivo de facilitar su análisis comparativo en términos de rendimiento, eficiencia y cobertura lingüística, se presentan agrupados de la siguiente forma:

Modelos compactos

Modelos optimizados para funcionar en entornos con recursos computacionales limitados (CPU o GPU modesta), generalmente mediante técnicas de distilación o reducción de parámetros. A la derecha de cada nombre se indica el identificador del modelo en Hugging Face, que puede utilizarse directamente para cargar el modelo desde la plataforma:

- **DistilBERT**: `distilbert-base-uncased`
- **TinyBERT**: `huawei-noah/TinyBERT_General_4L_312D`
- **MiniLM**: `nreimers/MiniLM-L6-H384-uncased`
- **MobileBERT**: `google/mobilebert-uncased`
- **ELECTRA-small**: `google/electra-small-discriminator`
- **DistilBETO**: `dccuchile/distilbert-base-spanish-uncased`

Modelos estándar

Modelos de tamaño base que ofrecen un buen equilibrio entre capacidad de representación semántica y coste computacional, adecuados para tareas complejas en entornos generales:

- **BERT**: `bert-base-uncased`
- **RoBERTa**: `roberta-base`
- **DeBERTa-v3-small**: `microsoft/deberta-v3-small`
- **ALBERT-base**: `albert-base-v2`
- **CamemBERT**: `camembert-base`
- **BETO**: `dccuchile/bert-base-spanish-wwm-uncased`
- **MarIA**: `PlanTL-GOB-ES/roberta-base-bne`

Modelos multilingües

Modelos entrenados con corpora en múltiples lenguas. Aunque presentan buena cobertura lingüística, su rendimiento puede verse afectado si no han sido específicamente optimizados para el español o para tareas de instrucción:

- **XLM-RoBERTa-base**: `xlm-roberta-base`
- **XLM-RoBERTa Large**: `xlm-roberta-large`

La elección de estos modelos responde a su amplia disponibilidad en la plataforma Hugging Face, su popularidad en la comunidad investigadora y su accesibilidad para su uso en entornos locales. Muchos de ellos cuentan con variantes optimizadas que permiten su ejecución eficiente en equipos sin acceso a grandes infraestructuras. Esta combinación de madurez tecnológica, soporte comunitario y eficiencia práctica los convierte en candidatos adecuados para tareas de clasificación en contextos reales con limitaciones técnicas.

Modelos ejecutados en LM Studio (formato `GGUF`)

En este estudio se ha utilizado una selección de modelos de lenguaje en formato `GGUF`, ejecutados de forma local mediante la herramienta LM Studio. Estos modelos se agrupan a continuación en tres categorías según su tamaño y características generales:

Modelos compactos

Modelos de baja capacidad diseñados para entornos con recursos computacionales limitados. Son adecuados para tareas básicas o situaciones en las que se prioriza la eficiencia. A la derecha de cada nombre se indica el archivo en formato `gguf` que puede cargarse directamente en *LM Studio*:

- **SmolLM 360M Instruct**: `smollm-360m-instruct-add-basics-q8_0.gguf`
- **Hermes 3 Llama 3.2B**: `Hermes-3-Llama-3.2-3B.Q4_K_M.gguf`

Modelos estándar

Modelos de tamaño medio (aproximadamente entre 7 y 13 mil millones de parámetros), que ofrecen un buen equilibrio entre rendimiento y consumo de recursos:

- **Mistral 7B Instruct**: `Mistral-7B-Instruct-v0.3-Q4_K_M.gguf`
- **Codestral 22B**: `Codestral-22B-v0.1-Q3_K_M.gguf`

Modelos multilingües

Modelos entrenados con datos en varios idiomas, diseñados para ofrecer cobertura lingüística amplia en tareas generales:

- **Qwen3 8B**: `Qwen3-8B-Q4_K_M.gguf`

Cada uno de estos modelos presenta particularidades en cuanto a tamaño, arquitectura y compatibilidad con entornos locales, lo que permite evaluar su comportamiento en distintas condiciones de uso. Su rendimiento se analiza con detalle en los capítulos dedicados a la experimentación.

Capítulo 3

Metodología de entrenamiento y evaluación de LLMs

3.1. Introducción

Los Modelos de Lenguaje de Gran Tamaño (LLMs) han demostrado una notable capacidad para abordar tareas de clasificación textual. En este capítulo se investiga su aplicación en un problema real: la categorización de descripciones comerciales de productos alimentarios dentro de un conjunto jerárquico de etiquetas del ámbito minorista.

La clasificación automática en este contexto plantea desafíos relevantes, como el desbalance de clases, la complejidad semántica de las etiquetas y las limitaciones computacionales del entorno de ejecución. Estos factores hacen especialmente pertinente comparar dos estrategias representativas: el ajuste fino supervisado (*fine-tuning*) y el aprendizaje sin reentrenamiento (*zero-shot learning*).

El objetivo de este estudio es explorar el potencial de ambos enfoques en escenarios prácticos, considerando no solo su rendimiento, sino también los recursos necesarios y la facilidad de implementación. Esta comparación busca aportar criterios útiles para la selección de técnicas en proyectos reales que involucren clasificación multiclase con LLMs.

3.2. Conjuntos de Datos Utilizados para el Entrenamiento

El conjunto de datos empleado en este estudio ha sido proporcionado por el Instituto Canario de Estadística (ISTAC), a partir de una versión estructurada del corpus originalmente desarrollado por la plataforma *SOYSuper*. La base de datos contiene información detallada sobre productos comercializados en supermercados españoles hasta el año 2022, donde cada fila representa un artículo individual obtenido de los catálogos digitales de distintas cadenas de distribución. El conjunto completo se encuentra disponible en el material complementario enlazado en el Apéndice A.

Las variables más relevantes del conjunto son las siguientes:

- **DESC_ART**: descripción original del artículo tal como figura en el supermercado, incluyendo abreviaturas, errores tipográficos y formatos comerciales habituales (por ejemplo, *"Leche semidesnatada 1L"*).
- **CATEGORIA**: categoría jerárquica del producto según la taxonomía interna de *SOYSuper*, que puede abarcar distintos niveles de especificidad, como *"Frescos y Charcutería Frutas Cítricos"*, *"Lácteos y Huevos Yogur Sabores"* o categorías generales como *"Congelados Helados y Hielo Helado"*.
- **DESC_PROD**: descripción estandarizada del producto, utilizada como etiqueta de referencia para la tarea de clasificación. Estas etiquetas no solo identifican el tipo genérico del

producto (por ejemplo, *"Patatas fritas"*), sino que también incluyen descriptores adicionales que especifican variantes relevantes, como el tipo de elaboración, el formato o el modo de presentación (por ejemplo, *"Aceite de oliva Virgen Extra"*, *"Jamón serrano vendido suelto"*, *"Sardinas en lata en aceite vegetal"*).

Estas etiquetas están estructuradas de forma sistemática y constan de dos componentes principales:

1. El **nombre del producto**, acompañado de **descriptores semánticos** que aportan información adicional sobre su presentación o contenido.
2. Un **sufijo de codificación** unido al nombre mediante un guion bajo (_) que indica restricciones sobre el tipo de marca asociada al producto.

Los sufijos incluidos al final de las etiquetas codifican información relevante definida por *SOY-Super*. Estos códigos permiten distinguir productos que, aunque similares en su denominación, difieren en el tipo de marca asociada. Su significado es el siguiente:

- `_SB`: el producto pertenece a una marca específica determinada (una o varias marcas concretas).
- `_WKB`: el producto es de marca conocida, aunque sin especificar cuál.
- `_BL`: el producto es de marca blanca o marca de distribuidor.
- `_NE`: el producto no tiene restricción sobre la marca; suele utilizarse para productos frescos o genéricos.

Gracias a estos sufijos, pueden distinguirse variantes de un mismo producto comercializado bajo condiciones distintas. Por ejemplo, `Arroz grano largo_WKB` y `Arroz grano largo_BL` representan el mismo producto desde el punto de vista funcional, pero se diferencian por su marca y posición en el mercado, lo cual puede ser relevante para ciertos análisis o recomendaciones.

El conjunto incluye un total de **312 etiquetas únicas** construidas siguiendo esta lógica. Sin embargo, su distribución es altamente desbalanceada: algunas etiquetas agrupan cientos de ejemplos, mientras que otras aparecen escasamente. Este fenómeno plantea desafíos significativos en el entrenamiento y evaluación de modelos, dado que tiende a favorecer las clases mayoritarias.

Algunas de las **etiquetas más frecuentes** son:

- `Aceite de oliva Virgen Extra_WKB` (708 ejemplos)
- `Infusión de flores, hierbas o frutas_WKB` (700 ejemplos)
- `Jamón serrano vendido suelto_NE` (594 ejemplos)
- `Potito de fruta o verduras_WKB` (546 ejemplos)
- `Patatas fritas_WKB` (533 ejemplos)

En el extremo opuesto, algunas **etiquetas minoritarias** están representadas por menos de 10 ejemplos:

- `Lechuga redonda_NE` (3 ejemplos)
- `Vino blanco de mesa en botella_BL` (3 ejemplos)

- `Barrita multicereales_NE` (6 ejemplos)
- `Sardinas en lata en aceite vegetal_BL` (6 ejemplos)

Este elevado grado de desbalance justifica el uso de métricas más sensibles como el F1-score macro o el análisis por clase individual, en lugar de confiar únicamente en la exactitud global. Además, la complejidad semántica de las etiquetas —que mezcla denominaciones genéricas, variantes de producto y codificación comercial— proporciona un entorno riguroso para evaluar la capacidad de los modelos de lenguaje de entender, representar y generalizar sobre descripciones reales y variadas.

Durante la fase de preprocesamiento se evaluó la conveniencia de conservar o eliminar determinadas variantes. En particular, las etiquetas con el sufijo `_NE`, al no aportar restricciones de marca, se consideraron prescindibles. En cambio, se decidió mantener las etiquetas con `_WKB`, `_BL` y `_SB`, al reflejar distinciones relevantes para análisis posteriores basados en estrategias de marca o posicionamiento comercial.

3.3. Ajuste fino (Fine-Tuning) de modelos preentrenados

El *fine-tuning* es una técnica esencial del aprendizaje transferido que permite adaptar un modelo de lenguaje previamente entrenado sobre grandes corpus generales —como Wikipedia, Common Crawl o libros digitalizados— a una tarea específica del mundo real. Esta adaptación se realiza mediante aprendizaje supervisado, utilizando un conjunto de datos etiquetados que representan ejemplos concretos del problema que se desea resolver.

Durante este proceso, el modelo conserva el conocimiento lingüístico general adquirido durante el preentrenamiento, pero modifica sus parámetros internos para especializarse en los patrones y estructuras del nuevo dominio. Como resultado, mejora significativamente su rendimiento en tareas particulares como la clasificación de textos, la detección de entidades o la generación controlada de lenguaje.

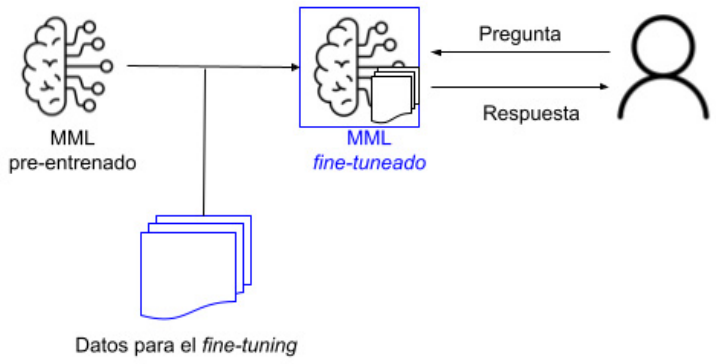

Figura 1: Esquema del proceso de ajuste fino de un MML. A partir de un modelo preentrenado, se incorporan ejemplos etiquetados del dominio objetivo para especializar el modelo. El resultado es un modelo afinado capaz de responder con mayor precisión en tareas específicas.

Por ejemplo, si se desea que un modelo aprenda a clasificar productos alimentarios, se le entrena con descripciones reales de artículos (como *"Yogur natural desnatado 0 % 125g"*) junto con su correspondiente etiqueta semántica (por ejemplo, `Yogur natural desnatado_WKB`). A través del ajuste fino, el modelo aprende a relacionar los patrones textuales presentes en las descripciones

con las categorías predefinidas, de modo que, una vez entrenado, pueda predecir automáticamente la etiqueta más adecuada para nuevas descripciones nunca vistas.

En este trabajo, se ha aplicado el ajuste fino a varios modelos preentrenados con el objetivo de resolver una tarea de clasificación multiclase sobre productos comercializados en supermercados españoles. El propósito es adaptar los modelos al dominio específico del comercio minorista, optimizando su capacidad para identificar etiquetas estandarizadas a partir de textos ruidosos, breves y no estructurados que simulan descripciones reales de catálogos digitales.

Preparación de los Datos y Estrategia de Entrenamiento

Para entrenar y evaluar los modelos se empleó una variante de validación cruzada con 4 *folds*, utilizando el conjunto de datos de productos de SOYSuper correspondiente hasta el año 2022. El procedimiento consistió en dividir los datos en 4 subconjuntos o *folds*. En cada iteración, tres de estos *folds* se utilizaron como conjunto de entrenamiento y el restante como conjunto de prueba (*test*), asegurando que todas las etiquetas presentes en el conjunto de prueba también estuvieran representadas en el de entrenamiento.

Este proceso se repitió 4 veces, alternando el *fold* asignado como conjunto de prueba en cada iteración. De esta forma se garantiza una evaluación más robusta y generalizable, evitando que los modelos se enfrenten a etiquetas nunca vistas durante el entrenamiento.

Es importante destacar que la división en *folds* se realizó de forma previa y se mantuvo fija para todos los modelos. Esto significa que cada modelo fue entrenado y evaluado exactamente con las mismas particiones de datos, lo que permite una comparación justa y directa entre arquitecturas bajo un marco experimental común.

El procedimiento seguido para el ajuste fino de todos los modelos evaluados fue el siguiente:

1. **Carga del modelo base**: se empleó la clase `AutoModelForSequenceClassification` de la librería `transformers`, que permite cargar modelos preentrenados desde Hugging Face y adaptarlos a tareas de clasificación multiclase. Para cada entrenamiento, se inicializó un modelo desde cero, ajustando el número de salidas al total de etiquetas del problema.

2. **Preparación de los datos de entrada**: se construyó un campo de texto unificado concatenando la descripción del artículo (`DESC_ART`) con su categoría (`CATEGORIA`), separados por el delimitador "—". Este texto se tokenizó con el `AutoTokenizer` correspondiente al modelo empleado. Las etiquetas (`DESC_PROD`) fueron codificadas de forma numérica mediante `LabelEncoder`.

3. **Estrategia de validación cruzada**: como ya se indicó, se utilizó validación cruzada manual con 4 particiones (*4-fold cross-validation*). Cada muestra del conjunto de datos participó una vez como test y tres veces como entrenamiento. Esta estrategia favorece la generalización y mejora la fiabilidad de las métricas obtenidas.

4. **Entrenamiento supervisado**: en cada fold, el modelo se entrenó desde cero durante **4 épocas**, utilizando el optimizador `AdamW` con regularización mediante `weight decay`. La tasa de aprendizaje se mantuvo constante y se utilizó la función de pérdida `CrossEntropyLoss`, apropiada para clasificación multiclase.

5. **Evaluación**: al finalizar cada entrenamiento, el modelo se evaluó sobre su correspondiente conjunto de prueba. Se calcularon métricas como **accuracy**, **precisión**, **recall** y **F1-score**, utilizando tanto el promedio *ponderado* (*weighted*) —que corrige el desequilibrio entre clases según su frecuencia— como el promedio *macro*, que otorga igual peso a cada clase sin importar su tamaño.

El uso combinado de métricas con promediado *macro* y *weighted* responde a la necesidad de evaluar tanto la equidad del modelo en todas las clases como su rendimiento global ajustado a la distribución real del conjunto de datos. Mientras que las métricas *macro* asignan el mismo peso a cada clase, permitiendo detectar deficiencias en aquellas menos representadas, las métricas *weighted* ponderan cada resultado según la frecuencia relativa de la clase correspondiente, proporcionando así una visión más representativa del comportamiento agregado.

Ejemplo ilustrativo. Supóngase que un modelo obtiene F1-scores de 0,90 para una clase A con 80 instancias, 0,60 para una clase B con 15 instancias, y 0,30 para una clase C con solo 5 instancias. El F1-score *macro*, que calcula el promedio simple de las tres clases, sería: (0,90 + 0,60 + 0,30)/3 = 0,60. En cambio, el F1-score *weighted*, que pondera por la proporción de ejemplos en cada clase, resultaría: 0,90×0,8 + 0,60×0,15 + 0,30×0,05 = 0,84. Este contraste evidencia cómo las métricas *macro* detectan mejor si el modelo falla en clases minoritarias, mientras que las *weighted* reflejan el rendimiento global más fielmente en contextos con desbalance.

La elección de entrenar durante **4 épocas** y utilizar el optimizador `AdamW` con `weight decay` respondió a criterios tanto empíricos como prácticos. En experimentos preliminares se observó que la mayoría de los modelos convergían antes de la quinta época, y prolongar el entrenamiento aumentaba el riesgo de sobreajuste sin mejorar significativamente el rendimiento. Además, dado que los experimentos se realizaron en entornos locales con recursos limitados (sin acceso a GPUs múltiples ni servidores de alto rendimiento), se priorizó una configuración computacionalmente eficiente que ofreciera un buen compromiso entre coste y rendimiento.

Las restricciones de hardware y su influencia sobre las decisiones metodológicas adoptadas se desarrollan con mayor detalle en la Sección 3.5 (*Limitaciones computacionales y justificación del enfoque*).

Reproducibilidad de los experimentos

Con el objetivo de garantizar la reproducibilidad, se tomaron las siguientes medidas:

- Se fijaron semillas aleatorias en las librerías `random`, `numpy` y `torch` antes de cada ejecución.
- Todos los modelos se entrenaron sobre el mismo conjunto de particiones de datos.
- Se mantuvieron constantes los hiperparámetros (épocas, tamaño de batch, función de pérdida, optimizador, etc.).
- Se almacenaron los modelos, métricas y codificadores en carpetas estructuradas por fold y arquitectura.

Ventajas del entrenamiento por *fine-tunning*

El ajuste fino presenta múltiples ventajas en tareas de clasificación específicas:

- **Adaptación al dominio**: mejora el rendimiento al aprender vocabulario y estructuras propias del contexto comercial.
- **Aprovechamiento de conocimiento previo**: reduce el tiempo y datos necesarios en comparación con un entrenamiento desde cero.
- **Resultados superiores**: los modelos afinados obtienen métricas significativamente mejores que versiones no ajustadas.

Limitaciones

No obstante, esta técnica también presenta algunas limitaciones:

- **Demanda computacional**: requiere recursos elevados, especialmente para modelos de gran tamaño.
- **Sobreajuste**: puede aprender patrones específicos si el conjunto de entrenamiento es reducido o poco variado.
- **Dependencia de etiquetas**: requiere datos anotados con precisión, lo cual puede ser costoso o difícil de conseguir.

3.4. Aprendizaje sin ajuste (Zero-shot learning)

El *zero-shot learning* es una estrategia que permite aplicar directamente un modelo de lenguaje preentrenado a una tarea concreta sin necesidad de realizar ningún ajuste adicional sobre sus parámetros. A diferencia del *fine-tuning*, donde el modelo es reentrenado con ejemplos etiquetados para especializarse en una tarea específica, en el enfoque *zero-shot* se reutilizan las capacidades generales adquiridas durante el preentrenamiento para abordar tareas nuevas sin entrenamiento adicional.

La diferencia fundamental radica en que el *fine-tuning* modifica internamente los pesos del modelo para adaptarlo al nuevo dominio, mientras que el *zero-shot* aprovecha únicamente el conocimiento ya adquirido, sin modificar su arquitectura ni su comportamiento interno.

Esta técnica se basa en la formulación de la tarea como una instrucción explícita en lenguaje natural (también llamada *prompt*), que el modelo debe interpretar. Por ejemplo, en una tarea de clasificación, se proporciona un texto y una lista de posibles categorías, pidiéndole al modelo que elija la más adecuada. El modelo razona sobre la tarea basándose en su conocimiento previo del lenguaje y del mundo, sin haber sido entrenado específicamente para ese problema.

En este trabajo, se ha aplicado el enfoque *zero-shot* utilizando modelos en formato `GGUF`, ejecutados localmente mediante el software `LM Studio`. Estos modelos han sido empleados sin reentrenamiento, evaluando directamente su capacidad para realizar tareas de clasificación a partir de instrucciones formuladas en lenguaje natural.

El enfoque *zero-shot* resulta especialmente útil cuando no se dispone de datos anotados, cuando se desea evaluar la capacidad general de razonamiento del modelo, o cuando se requiere una solución inmediata sin coste computacional asociado al reentrenamiento.

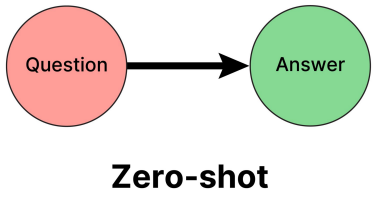

Figura 2: Esquema conceptual del aprendizaje *zero-shot*. El modelo preentrenado es capaz de generar respuestas a partir de instrucciones directamente formuladas en lenguaje natural, sin haber sido afinado previamente para esa tarea específica.

Preparación de los datos y estrategia de evaluación

Este trabajo evaluó la capacidad de distintos modelos de lenguaje para abordar una tarea de clasificación de productos en modo *zero-shot*, ejecutándolos localmente a través de la herramienta LM Studio y utilizando el formato `GGUF`. La tarea consistía en presentar al modelo una breve descripción del producto y pedirle que identificara la categoría correcta entre un conjunto limitado de opciones.

El planteamiento se fundamentó en la técnica de **ingeniería del prompt** (*prompt engineering*), especialmente relevante en contextos sin ajuste previo, donde la formulación precisa de la instrucción incide de manera directa en la calidad de la inferencia. En este caso, se empleó una plantilla en inglés que instruía al modelo a seleccionar una única categoría de una lista dada y devolver únicamente su nombre, sin ningún comentario adicional.

Por ejemplo, ante un producto como *"Yogur natural desnatado"*, el *prompt* generado era:

> *"Given the product description: 'Yogur natural desnatado', which of the following categories does it belong to? Choose only one: Dairy, Bakery, Beverages, Preserves, Fruits and vegetables."*

Esta plantilla se completaba dinámicamente con la descripción del producto y las etiquetas correspondientes al subconjunto en cuestión, dando lugar al *prompt* final. Todo el proceso —desde la generación automática de instrucciones hasta la ejecución de inferencias y el cálculo de métricas— fue automatizado mediante un script, lo que permitió evaluar de forma sistemática, reproducible y coherente el rendimiento de los modelos en modo *zero-shot*.

Durante la experimentación se observó que incluir un número excesivo de categorías en una sola instrucción afectaba negativamente a la precisión, favoreciendo respuestas sesgadas hacia las primeras opciones. Para mitigar este efecto, el conjunto de evaluación se segmentó en **32 subconjuntos**, cada uno compuesto por **10 etiquetas** agrupadas por afinidad semántica. Esta estrategia redujo la carga contextual del modelo y favoreció una inferencia más precisa.

Además, se observó cierta heterogeneidad en el formato de las respuestas generadas. Algunos modelos devolvían frases completas, otros reformulaciones parciales o etiquetas con ligeras variaciones. Para estandarizar la evaluación, se aplicó un proceso de normalización textual previa a la comparación con la etiqueta esperada, asegurando la coherencia de las métricas sin depender estrictamente del formato exacto devuelto por el modelo.

Por último, se utilizaron las mismas métricas clásicas que en el caso con ajuste fino: **precisión** (`precision`), **exhaustividad** (`recall`) y **F1-score** , calculadas según dos estrategias de promediado: *macro* y *weighted*.

Reproducibilidad de los experimentos

Para garantizar la trazabilidad y replicabilidad del proceso:

- Se definieron y almacenaron los 32 subconjuntos de evaluación con combinaciones fijas de 10 etiquetas cada uno.
- Se utilizó un mismo formato de instrucción para todos los modelos, adaptando únicamente el lenguaje si el modelo requería un estilo específico (por ejemplo, en inglés o español).
- Se registraron todas las predicciones generadas por cada modelo y se guardaron junto con las etiquetas reales y los subconjuntos correspondientes.
- Se implementaron funciones automáticas de limpieza y evaluación para asegurar consistencia en el tratamiento de las respuestas y en el cálculo de métricas.

Este procedimiento permitió evaluar de forma sistemática la capacidad de distintos modelos generativos para realizar clasificación de productos en escenarios realistas, sin necesidad de entrenamiento adicional.

Fortalezas del enfoque *zero-shot*

Las principales fortalezas del enfoque *zero-shot* son:

- No requiere entrenamiento adicional ni anotación previa.
- Permite una aplicación inmediata a nuevas tareas o dominios.
- Es útil en contextos donde no se dispone de suficientes datos etiquetados o se requiere adaptabilidad.

Limitaciones

- El rendimiento es generalmente inferior al obtenido mediante técnicas de *fine-tuning*, especialmente en tareas especializadas.
- Alta sensibilidad a la formulación del *prompt*, lo que requiere experimentación y afinado manual.
- Cuando el número de clases es elevado, el modelo puede perder precisión por sobrecarga de contexto, como se observó en este estudio.
- Requiere postprocesamiento para adaptar las respuestas libres del modelo a un formato de clase discreta.

3.5. Limitaciones computacionales y justificación del enfoque

Este trabajo se ha desarrollado íntegramente en entornos locales, sin acceso a servidores de alto rendimiento ni GPUs avanzadas, lo que condicionó la elección de modelos, configuración de entrenamiento y número de épocas.

Se optó por modelos de tamaño medio como `roberta-base`, `deberta-v3-small` o `xlm-roberta-base`, por su buen equilibrio entre capacidad y eficiencia. El entrenamiento se limitó a **4 épocas**, ya que pruebas preliminares indicaron convergencia antes de la quinta, evitando sobreajuste y consumo innecesario de recursos. Se usaron tamaños de batch reducidos, la función `CrossEntropyLoss` y el optimizador `AdamW`, adecuados para entornos limitados y tareas multiclase.

Para el enfoque *zero-shot* se recurrió a modelos cuantizados en formato `GGUF`, ejecutados en CPU mediante `LM Studio`, permitiendo evaluar LLMs recientes sin infraestructura avanzada.

Lejos de ser una desventaja, esta restricción permite validar la viabilidad del uso de LLMs en contextos realistas como pequeñas empresas, proyectos académicos o desarrollos individuales. El enfoque empleado refuerza el carácter práctico, replicable y sostenible del estudio, demostrando que es posible aplicar metodologías rigurosas con recursos modestos.

Capítulo 4

Análisis de los resultados del problema de clasificación

4.1. Introducción

Este capítulo presenta los resultados obtenidos al aplicar modelos de lenguaje preentrenados (LLMs) a una tarea de clasificación de productos a partir de descripciones textuales. Se han considerado dos enfoques complementarios: el ajuste fino (*fine-tuning*), donde los modelos se entrenan con ejemplos etiquetados, y el aprendizaje sin ajuste (*zero-shot*), que evalúa la capacidad de inferencia directa sin reentrenamiento, utilizando herramientas como LM Studio. El análisis compara distintas arquitecturas y tamaños de modelo en términos de precisión, cobertura y eficiencia, con el objetivo de identificar configuraciones óptimas para contextos prácticos con recursos computacionales limitados.

4.2. Análisis por métricas de clasificación

En esta sección se analizan en profundidad los resultados de los modelos mediante el uso de métricas estándar de evaluación en tareas de clasificación multiclase. El objetivo es valorar con mayor detalle el comportamiento predictivo de cada arquitectura, tanto en contextos con *fine-tuning* como en configuraciones *zero-shot*. Para ello, se comparan métricas como la **precisión**, la **exhaustividad**, el **F1-score** y la **exactitud**, utilizando tanto promedios *macro* —que tratan por igual a todas las clases— como promedios *weighted* (ponderados) —que tienen en cuenta la frecuencia de cada clase.

Este análisis permite identificar no solo qué modelos obtienen mejores resultados globales, sino también cuáles son más equilibrados en su rendimiento entre clases frecuentes y minoritarias. Además, se complementa la evaluación con visualizaciones comparativas y desgloses individuales de cada métrica para facilitar la interpretación.

Evaluación de modelos entrenados mediante ajuste fino

La Tabla 4.1 y la Tabla 4.2 presentan un resumen comparativo de los resultados obtenidos por cada modelo, incluyendo la exactitud general (*accuracy*) y las métricas clásicas de clasificación: precisión(*precision*), exhaustividad(*recall*) y F1-score.

Cuadro 4.1: Resumen de métricas `macro` por modelo

Modelo	**Exactitud**	**F1$_{\text{macro}}$**	**Precisión$_{\text{macro}}$**	**Recall$_{\text{macro}}$**
`albert`	0.556	0.276	0.272	0.313
`bert`	0.913	0.807	0.815	0.818
`beto`	0.924	0.857	0.869	0.864
`camembert`	0.865	0.642	0.642	0.670
`deberta`	0.865	0.666	0.678	0.686
`distilbert`	0.921	0.843	0.854	0.849
`distilbeto`	0.922	0.846	0.862	0.852
`electra`	0.787	0.449	0.453	0.492
`maria`	0.920	0.847	0.862	0.851
`minilm`	0.760	0.385	0.380	0.435
`mobilebert`	0.912	0.850	0.872	0.848
`roberta`	0.895	0.759	0.777	0.768
`tinybert`	0.546	0.175	0.176	0.228
`xlmroberta`	0.456	0.351	0.354	0.363
`xlmroberta_large`	0.027	$8{,}7 \times 10^{-5}$	$1{,}77 \times 10^{-4}$	0.003

Cuadro 4.2: Resumen de métricas `weighted` por modelo

Modelo	**Exactitud**	**F1$_{\text{weighted}}$**	**Precisión$_{\text{weighted}}$**	**Recall$_{\text{weighted}}$**
`albert`	0.556	0.473	0.446	0.556
`bert`	0.913	0.900	0.897	0.913
`beto`	0.924	0.915	0.916	0.924
`camembert`	0.865	0.833	0.824	0.865
`deberta`	0.865	0.841	0.837	0.865
`distilbert`	0.921	0.911	0.911	0.921
`distilbeto`	0.922	0.913	0.915	0.922
`electra`	0.787	0.735	0.722	0.787
`maria`	0.920	0.912	0.914	0.920
`minilm`	0.760	0.697	0.675	0.760
`mobilebert`	0.912	0.906	0.910	0.912
`roberta`	0.895	0.878	0.875	0.895
`tinybert`	0.546	0.447	0.433	0.546
`xlmroberta`	0.456	0.431	0.426	0.456
`xlmroberta_large`	0.027	0.001	0.001	0.027

Para complementar la evaluación global, se presenta a continuación un análisis individualizado de las principales métricas de clasificación. Este enfoque permite identificar fortalezas y debilidades específicas de los modelos evaluados, considerando tanto la versión *macro* de cada métrica (que trata por igual a todas las clases) como la versión *ponderada* (*weighted*), que da mayor peso a las clases más frecuentes.

Comenzamos con el análisis de la **precisión** (*precision*), que indica la proporción de predicciones correctas entre todas las realizadas para cada clase. En la Figura 1 se muestra una comparativa de los distintos modelos. Los modelos entrenados específicamente en español —`beto`, `distilbeto` y `maria`— obtienen las puntuaciones más altas, superando el 86 % tanto en la versión *macro* como *weighted*. Les siguen de cerca `distilbert`, `mobilebert` y `bert`. Por el contrario, `albert`, `tinybert`, `minilm` y `xlmroberta_large` muestran los niveles más bajos de precisión, lo que revela una mayor propensión a cometer errores de clasificación.

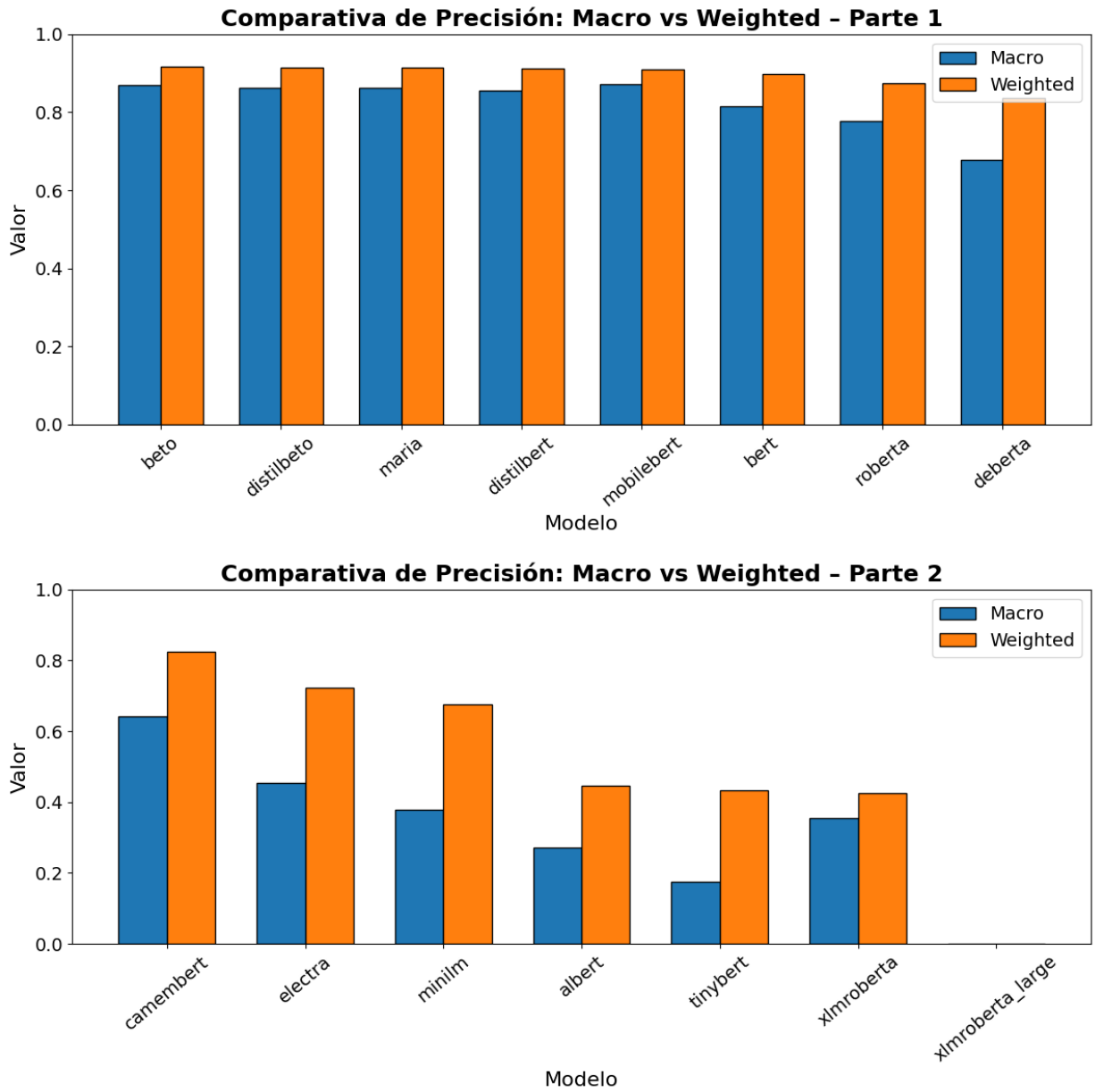

Figura 1: Comparativa de precisión por modelo, en promedio *macro* y *weighted*.

Seguimos con la evaluación de la **exhaustividad** (*recall*), que mide la capacidad de los modelos para recuperar correctamente todas las instancias relevantes, es decir, cuántos de los elementos de una clase son correctamente identificados. En la Figura 2, se observa que los modelos entrenados específicamente en español —`beto`, `distilbeto` y `maria`— obtienen los valores más altos tanto en promedio *macro* como *ponderado*.

Entre los modelos generalistas, `bert` y `distilbert` también muestran un rendimiento sólido, con puntuaciones que superan el 80 %. En el otro extremo, `tinybert`, `albert` y especialmente `xlmroberta_large` obtienen resultados significativamente más bajos, sobre todo en la versión *macro*, lo que evidencia su dificultad para cubrir correctamente las clases menos frecuentes, posiblemente debido a su menor capacidad de generalización o a una adaptación inadecuada al dominio.

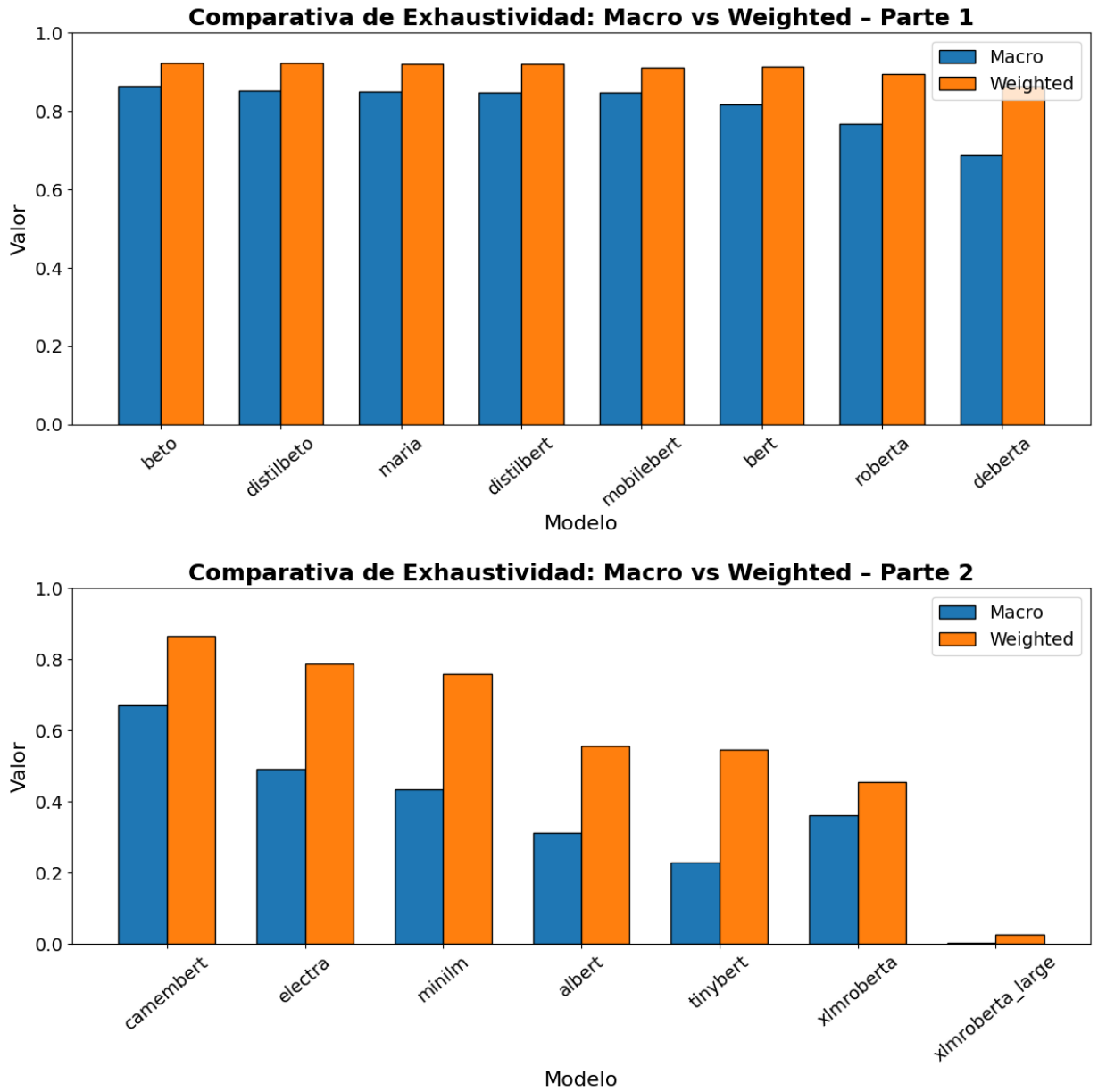

Figura 2: Comparativa de *recall* por modelo, en promedio *macro* y *weighted*.

Continuamos con el análisis del **F1-score**, una métrica que combina la precisión y la exhaustividad en una única medida armónica, proporcionando así una visión más equilibrada del rendimiento de los modelos. Esta métrica es especialmente útil cuando se trabaja con clases desbalanceadas, ya que penaliza los desequilibrios entre falsos positivos y falsos negativos.

Tal como se muestra en la Figura 3, los modelos entrenados en español —`beto`, `distilbeto` y `maria`— lideran nuevamente en rendimiento, alcanzando valores superiores al 90 % en su versión *weighted* y manteniéndose por encima del 85 % en promedio *macro*. Estos resultados reflejan una excelente capacidad para clasificar correctamente tanto clases frecuentes como minoritarias.

Otros modelos como `distilbert`, `mobilebert` y `bert` también muestran un rendimiento competitivo. Sin embargo, modelos como `albert`, `tinybert` y especialmente `xlmroberta_large` exhiben valores muy bajos, en particular en la métrica *macro*, lo cual evidencia serias dificultades para mantener un buen equilibrio en la clasificación de todas las clases presentes.

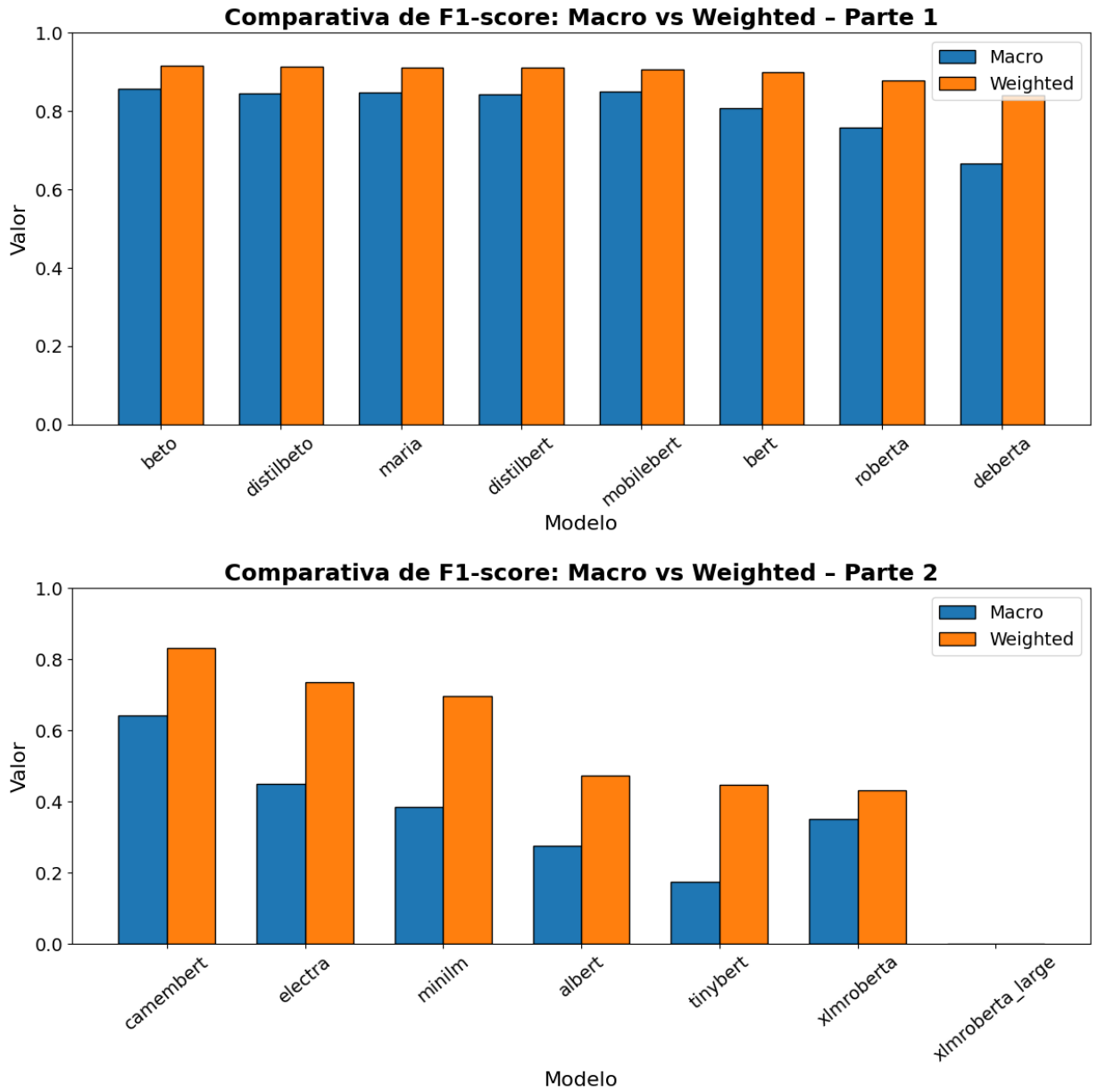

Figura 3: Comparativa de F1-score por modelo, en promedio *macro* y *weighted*.

Finalizamos este análisis individual con la **exactitud** (*accuracy*), una métrica ampliamente utilizada por su simplicidad, ya que representa el porcentaje total de aciertos del modelo sobre el conjunto de evaluación. No obstante, su interpretación debe hacerse con cautela, especialmente en escenarios donde existen clases mayoritarias que pueden sesgar el resultado global.

En la Figura 4 se observa que la mayoría de los modelos obtienen valores elevados de exactitud, destacando una vez más aquellos específicamente entrenados en español —`beto`, `distilbeto` y `maria`— que superan el 92 %. Modelos multilingües como `roberta`, `bert` y `mobilebert` también alcanzan cifras destacadas.

Sin embargo, esta métrica debe contrastarse con otras como *recall* y F1-score *macro*, ya que un *accuracy* alto puede ocultar un rendimiento deficiente en las clases minoritarias. Este es el caso de modelos como `electra` o `deberta`, que muestran un buen valor global pero presentan desequilibrios internos importantes. Por otro lado, el modelo `xlmroberta_large` obtiene una exactitud

prácticamente nula, lo que evidencia una completa falta de adaptación al dominio de la tarea.

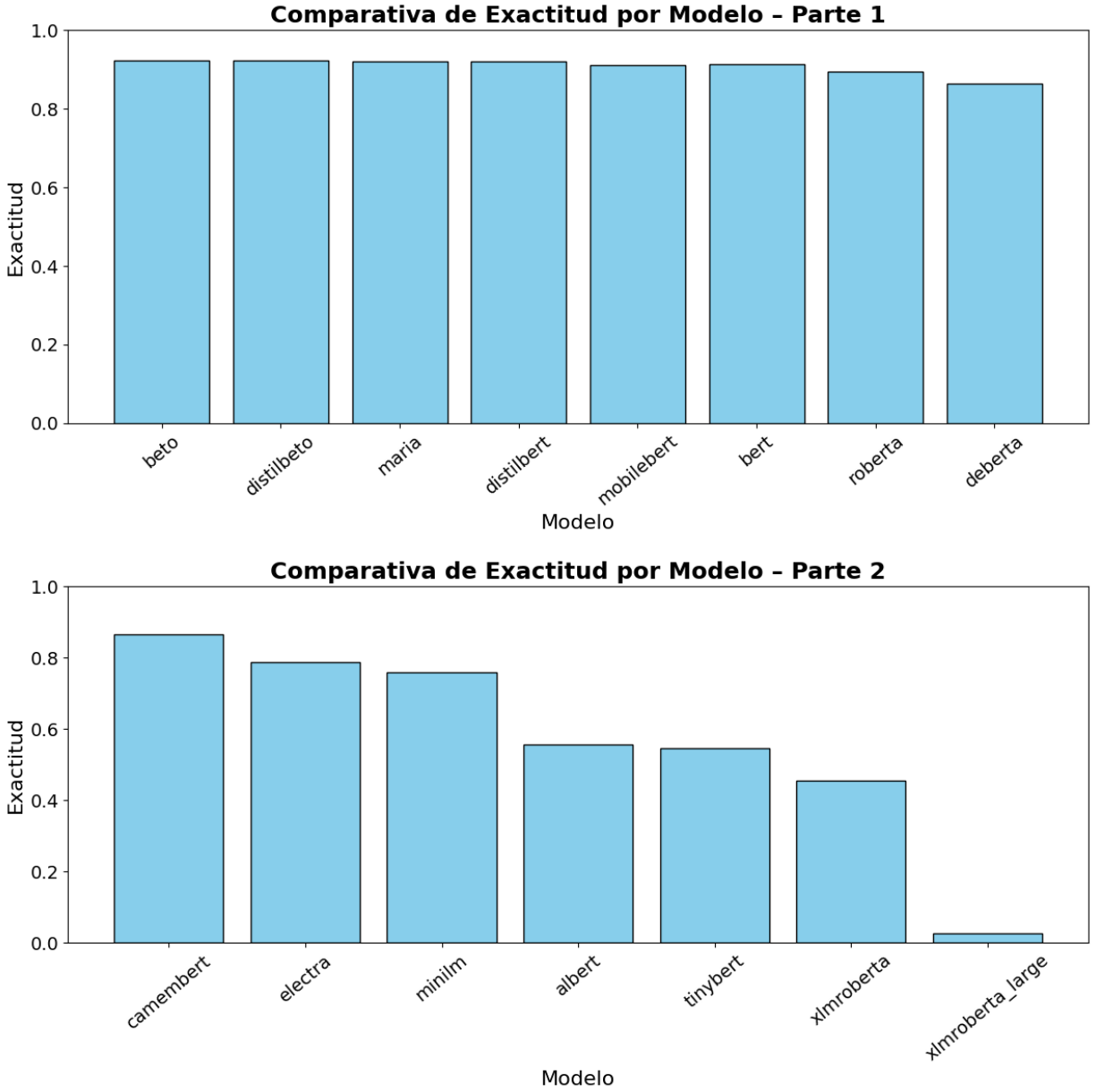

Figura 4: Comparativa de exactitud por modelo.

Validación en escenarios *zero-shot*

Las Tablas 4.3 y 4.4 presentan un resumen de las métricas clásicas de clasificación —precisión, exhaustividad y F1-score— en promedio *macro* y *weighted*, así como la exactitud general (*accuracy*) para cada modelo evaluado en modo *zero-shot*.

Cuadro 4.3: Métricas macro promedio por modelo en *zero-shot*

Modelo	**Precisión$_{\text{macro}}$**	**Recall$_{\text{macro}}$**	**F1$_{\text{macro}}$**
`CodeStral`	0.1718	0.1042	0.1014
`Hermes`	0.0243	0.0081	0.0092
`Mistral`	0.2198	0.1832	0.1851
`Qwen`	0.5239	0.4925	0.4709
`SmolLM`	0.0034	0.0012	0.0013

Cuadro 4.4: Métricas ponderadas y exactitud por modelo en *zero-shot*

Modelo	**Precisión$_{\text{weighted}}$**	**Recall$_{\text{weighted}}$**	**F1$_{\text{weighted}}$**	**Exactitud**
`CodeStral`	0.6732	0.2811	0.3124	0.2811
`Hermes`	0.4623	0.0979	0.1218	0.0979
`Mistral`	0.8187	0.6262	0.6608	0.6262
`Qwen`	0.8172	0.7023	0.7014	0.7023
`SmolLM`	0.0843	0.0177	0.0224	0.0177

Como en la evaluación de los modelos entrenados mediante ajuste fino, a continuación se inicia el análisis del rendimiento en el escenario *zero-shot* comenzando por la métrica de **precisión** (*precision*).

En la Figura 5 se observa que el modelo `Qwen` obtiene los mejores resultados tanto en precisión *macro* como *weighted*, evidenciando un buen equilibrio entre clases frecuentes e infrecuentes. Le sigue `Mistral`, con un rendimiento competitivo en precisión ponderada, aunque con una marcada caída en su promedio *macro*, lo que sugiere un sesgo hacia las clases mayoritarias.

Modelos como `CodeStral` y `Hermes` muestran un rendimiento claramente desigual: la precisión ponderada se mantiene en niveles aceptables, pero la versión *macro* revela importantes deficiencias en la clasificación de clases menos representadas. Finalmente, `SmolLM` presenta los resultados más bajos en ambas métricas, lo que indica una capacidad predictiva muy limitada en el escenario sin ajuste.

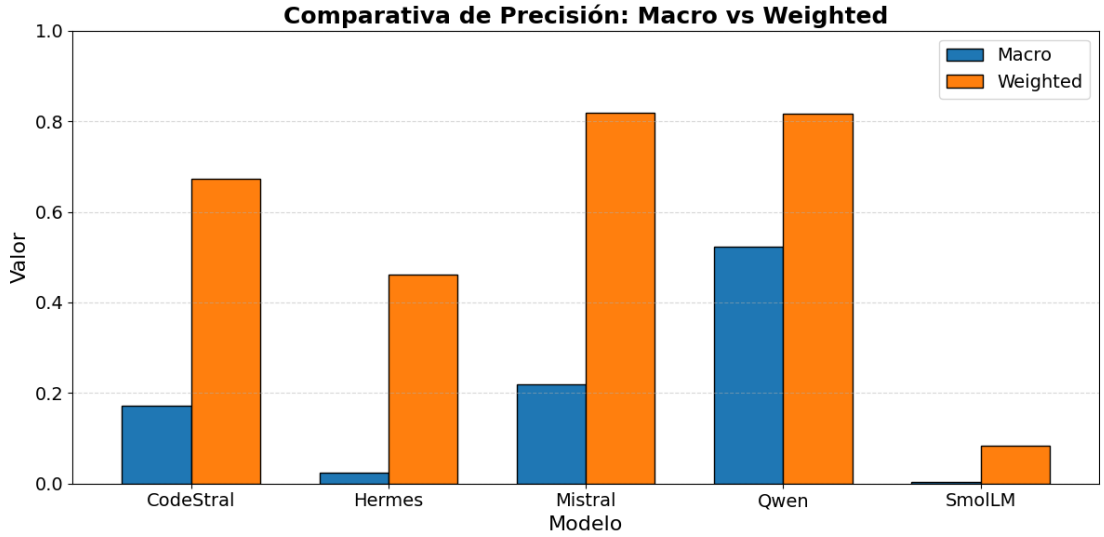

Figura 5: Comparativa de precisión por modelo: *macro* vs. *weighted*.

Como continuación del análisis en el escenario *zero-shot*, se examina la métrica de **exhaus-**

tividad (*recall*), que indica la capacidad del modelo para recuperar correctamente las instancias relevantes.

En la Figura 7 se aprecia que `Qwen` lidera con claridad, logrando una buena cobertura tanto en clases frecuentes como infrecuentes. Le sigue `Mistral`, aunque con un descenso notable en la media *macro*, lo que indica cierto sesgo hacia las clases mayoritarias.

`CodeStral` obtiene valores moderados, mientras que `Hermes` y `SmolLM` presentan un rendimiento muy limitado, con una recuperación escasa incluso en las clases más comunes.

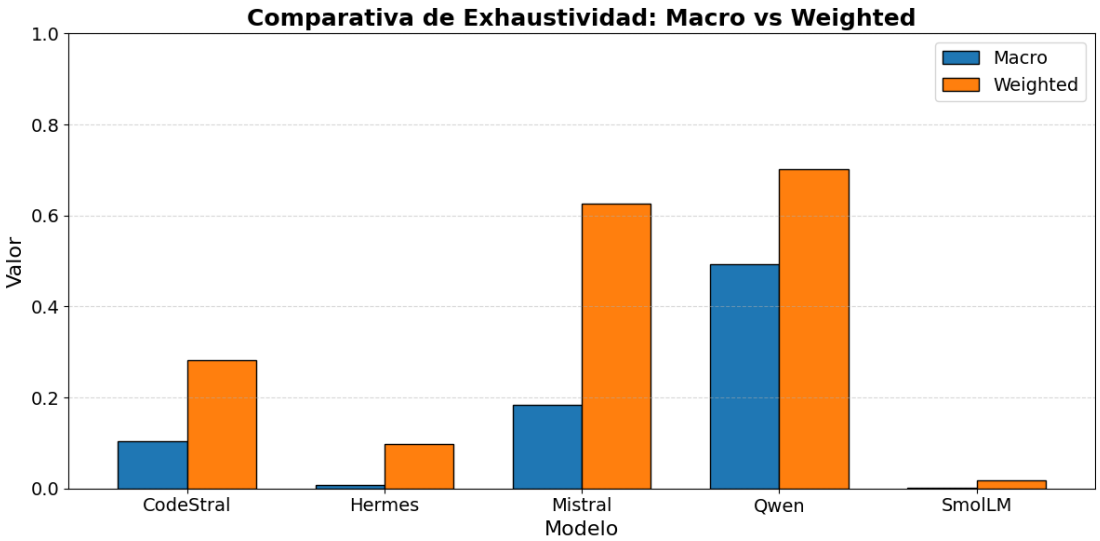

Figura 6: Comparativa de *recall* por modelo: *macro* vs. *weighted*.

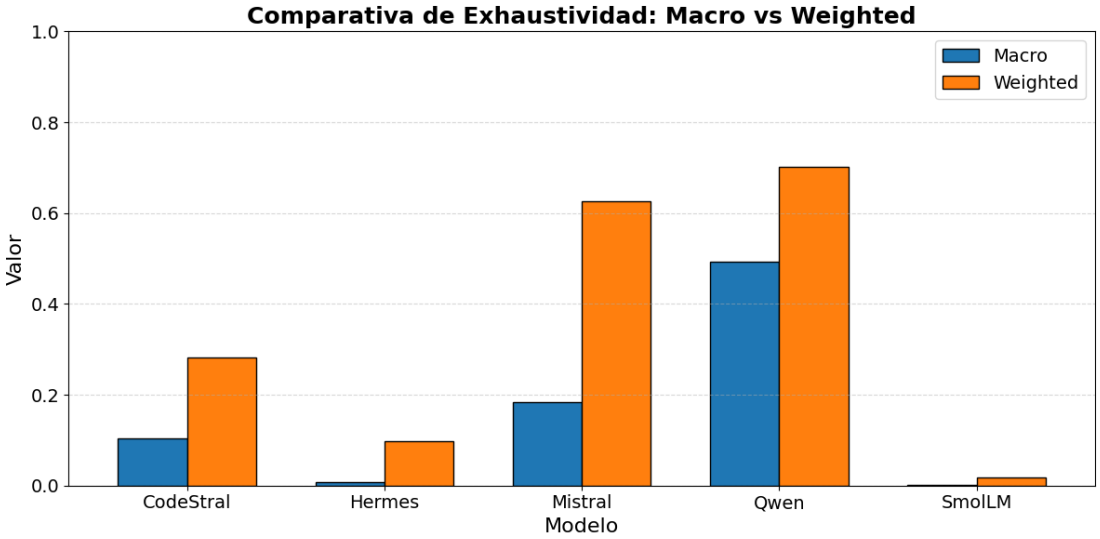

Figura 7: Comparativa de *recall* por modelo: *macro* vs. *weighted*.

Continuando con la evaluación en el escenario *zero-shot*, se analiza ahora el **F1-score**, que combina precisión y exhaustividad en una única métrica, especialmente útil en contextos desbalanceados.

En la Figura 8 se observa que `Qwen` obtiene los mejores resultados tanto en promedio *weighted* como *macro*, lo que evidencia su capacidad para mantener un rendimiento equilibrado. `Mistral`

le sigue con un buen resultado ponderado, aunque con una caída más acusada en la media simple, señalando un sesgo hacia clases frecuentes.

`CodeStral` muestra un desempeño moderado, mientras que `Hermes` y `SmolLM` presentan valores muy bajos, sin capacidad para equilibrar correctamente precisión y recall.

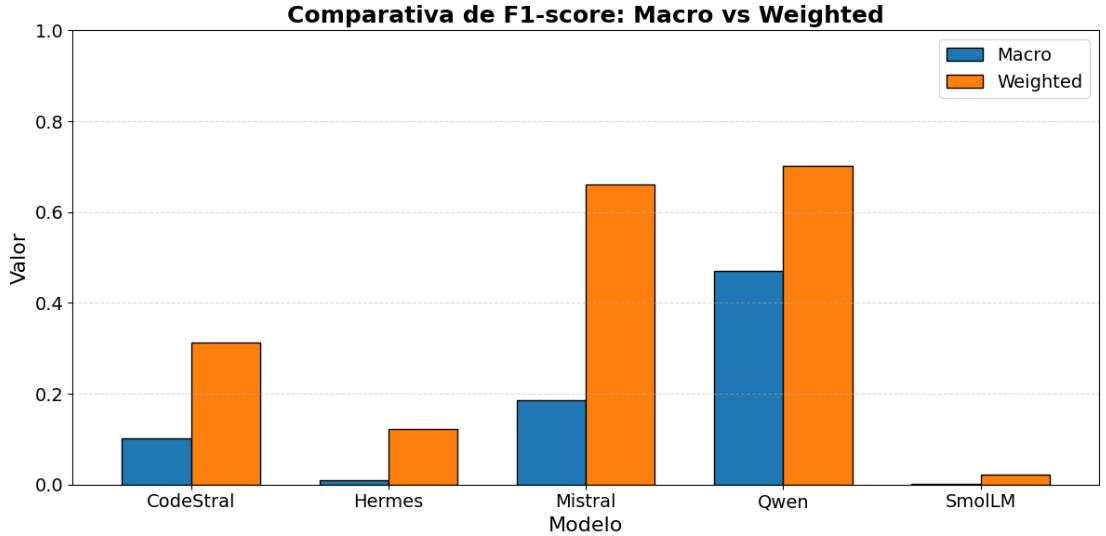

Figura 8: Comparativa de F1-score por modelo: *macro* vs. *weighted*.

Adicionalmente, se examina la métrica de **exactitud** (*accuracy*), que refleja el porcentaje de aciertos exactos, aunque puede resultar poco fiable en entornos con clases desbalanceadas.

En la Figura 9 se observa que `Qwen` alcanza la mayor exactitud (cerca del 70 %), seguido de `Mistral` (63 %), lo que confirma su buen desempeño general. `CodeStral` presenta un resultado moderado (29 %), mientras que `Hermes` y especialmente `SmolLM` quedan muy por debajo, con valores del 10 % y 2 % respectivamente.

Dado que esta métrica no contempla el equilibrio entre clases, su interpretación debe complementarse con otras medidas como el *recall* o el F1-score *macro* para evaluar adecuadamente el rendimiento en escenarios con fuerte desbalance.

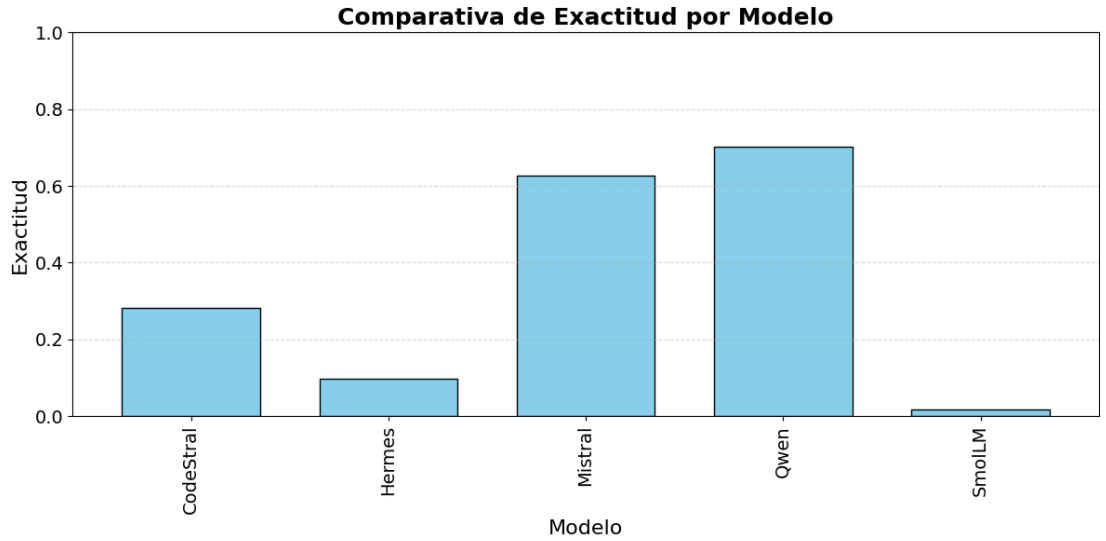

Figura 9: Comparativa de exactitud (*accuracy*) por modelo.

4.3. Análisis por grupo de modelos y arquitectura para modelos entrenados mediante *fine-tuning*

Esta sección examina el rendimiento de los modelos ajustados agrupándolos según su tamaño y arquitectura. Se comparan tres categorías funcionales (compactos, estándar y multilingües) y se distingue entre arquitecturas derivadas de BERT y otras más innovadoras, con el fin de identificar configuraciones eficientes y consistentes en tareas de clasificación multiclase.

Agrupación arquitectónica y análisis de rendimiento

En el escenario de ajuste fino, todos los modelos comparten una misma **arquitectura basada en Transformers**, aunque difieren en aspectos clave como el tamaño, el idioma predominante del preentrenamiento o el grado de compresión. Para facilitar la comparación, se agruparon funcionalmente en tres categorías: **compactos**, **estándar** y **multilingües**.

La Figura 10 presenta el rendimiento medio por grupo de modelos, evidenciando que los modelos **estándar** (trazo naranja) superan a los demás en todas las métricas, alcanzando valores cercanos o superiores al 0,85 en **exactitud**, **precisión** y **exhaustividad** *weighted*. Los **compactos** (trazo azul) se mantienen competitivos, con una ligera caída en métricas *macro*, lo que refleja una menor cobertura de clases minoritarias. En contraste, los modelos **multilingües** (trazo verde) presentan el peor desempeño global, con **exactitud** en torno al 0,30 y **F1-score** *macro* por debajo del 0,18.

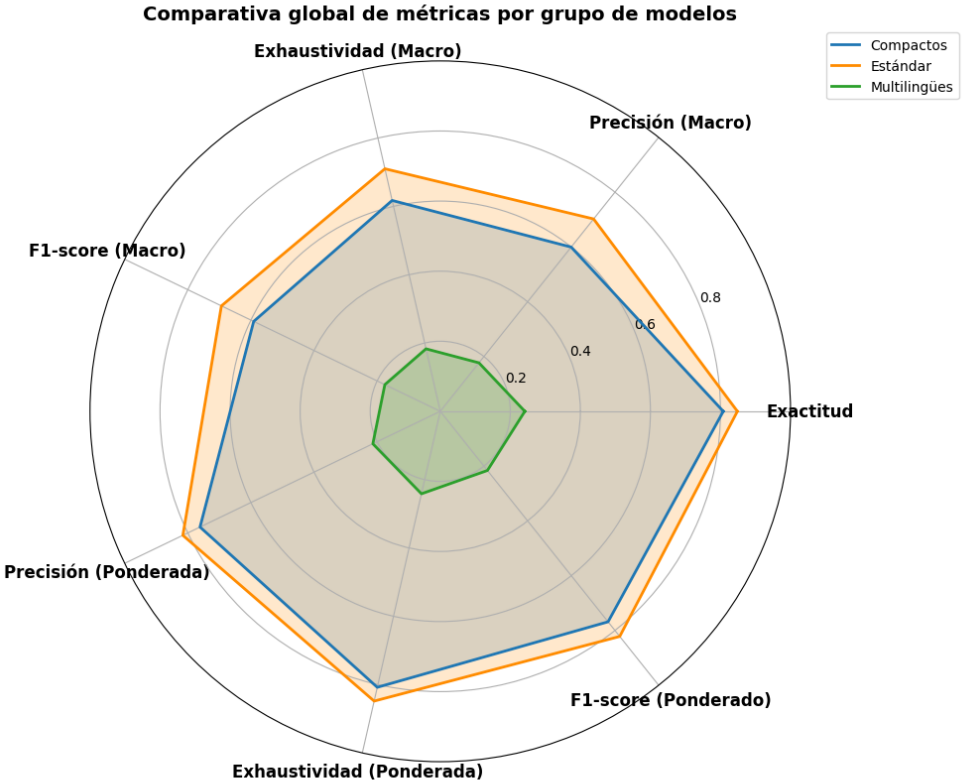

Figura 10: Rendimiento medio por grupo de modelos tras ajuste fino.

Este análisis sugiere que el tamaño completo de arquitectura (*base*) ofrece la mejor relación coste–beneficio, mientras que los modelos compactos son una alternativa viable en entornos con

recursos limitados. En cambio, el uso de modelos multilingües sin ajuste específico al dominio puede perjudicar seriamente el rendimiento.

Para valorar la consistencia y estabilidad entre modelos, se recurre a gráficos de caja. La Figura 11 muestra la distribución de métricas *macro* y **exactitud**, donde los grupos **estándar** y **compactos** mantienen alta concentración de valores, mientras que los **multilingües** revelan una alta dispersión y valores mínimos muy bajos. La Figura 12, centrada en métricas *weighted*, muestra una menor brecha entre grupos, aunque los modelos multilingües siguen presentando mayor variabilidad.

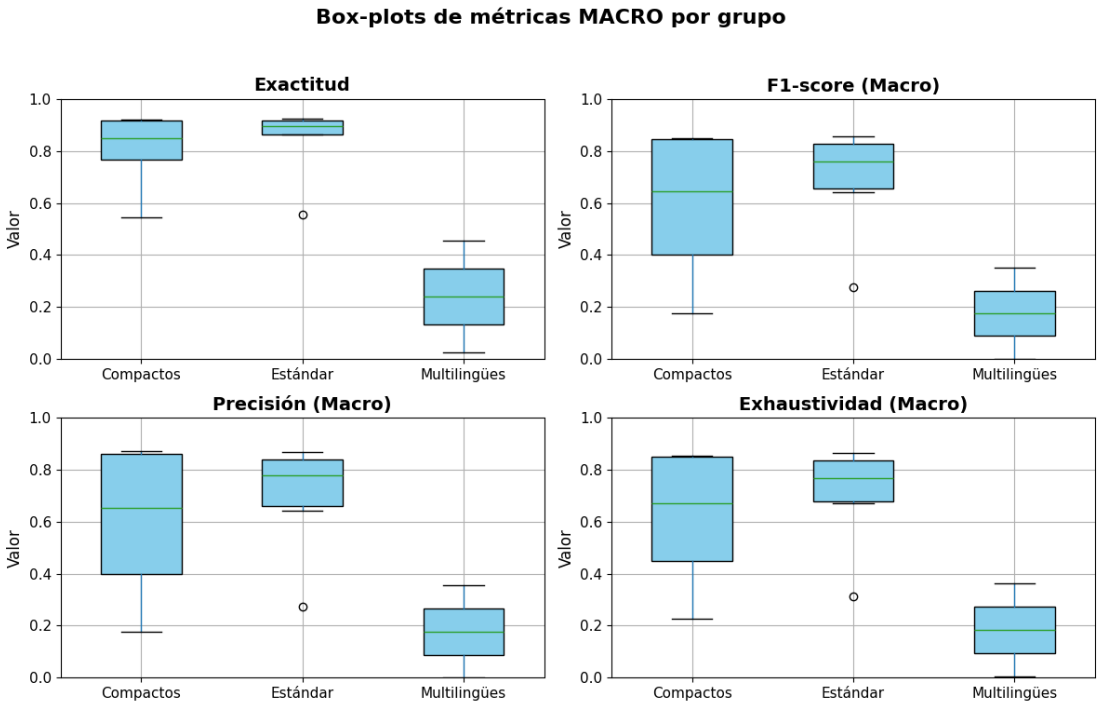

Figura 11: Distribución de métricas *macro* y **exactitud** por grupo funcional de modelos.

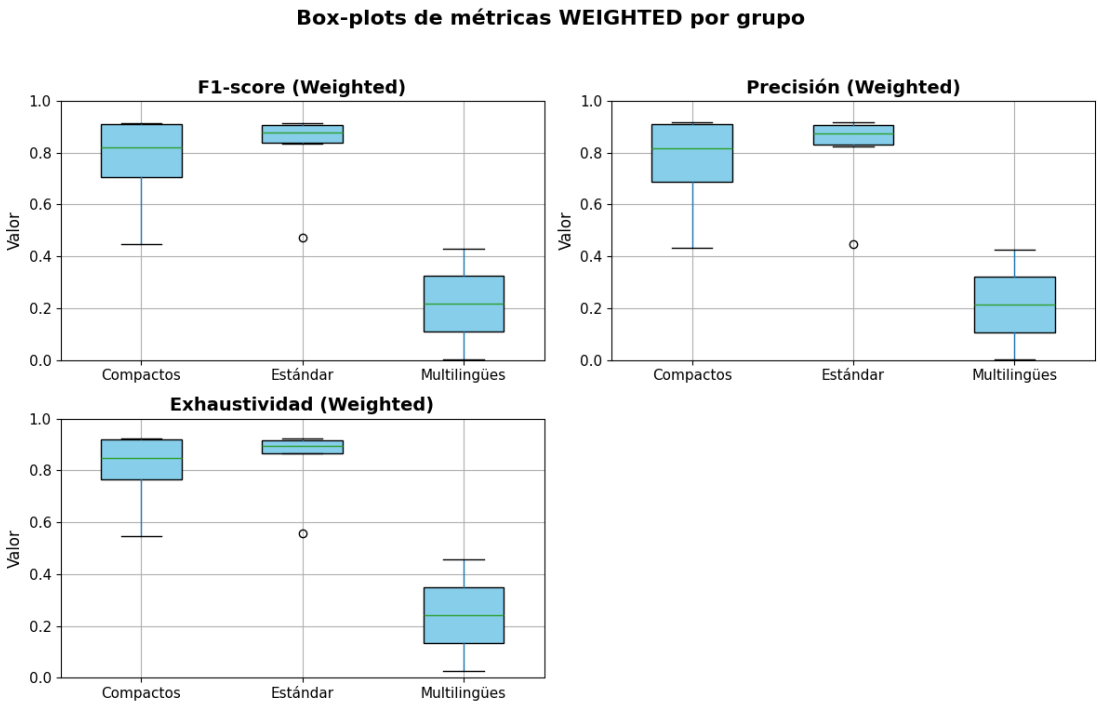

Figura 12: Distribución de métricas *weighted* por grupo funcional de modelos.

Cabe destacar un valor atípico dentro del grupo **estándar**, correspondiente al modelo `Albert`, cuyo bajo rendimiento genera una mayor dispersión interna.

Arquitecturas derivadas de BERT vs innovadoras

Más allá del agrupamiento funcional, se examinó el impacto de la arquitectura base. Los modelos se clasificaron como **BERT-like** —incluyendo variantes `DistilBERT`, adaptaciones monolingües como `BETO`, `DistilBETO` y `Maria`, y modelos tipo `RoBERTa`— frente a **arquitecturas innovadoras**, como `TinyBERT`, `ALBERT`, `MiniLM`, `MobileBERT`, `ELECTRA`, `DeBERTa` o `XLM-RoBERTa`, que presentan variantes multilingües o estructuras de atención no convencionales. La Figura 13 confirma que los modelos **BERT-like** superan consistentemente a los innovadores, con especial ventaja en métricas *macro*, lo que sugiere una mejor cobertura de clases minoritarias.

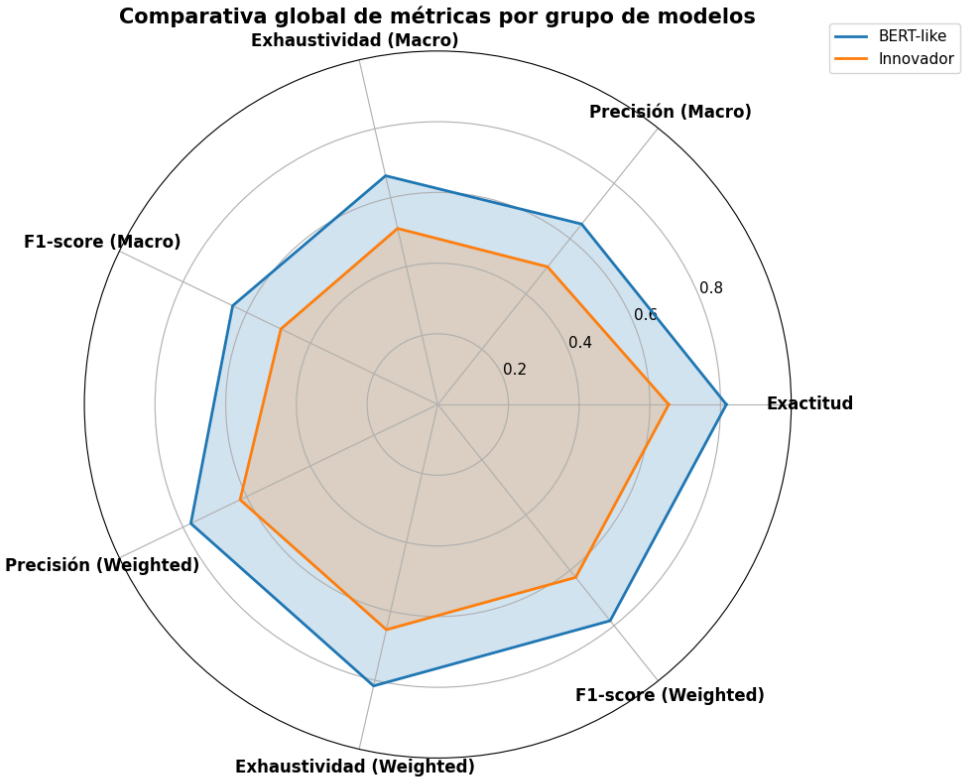

Figura 13: Rendimiento medio por tipo de arquitectura tras ajuste fino.

Las Figuras 14 y 15 refuerzan esta conclusión: los modelos **BERT-like** presentan mejores medianas y menor dispersión, especialmente en métricas *macro*, donde la consistencia es clave en tareas multiclase. Las diferencias se atenúan en las métricas *weighted*, aunque siguen siendo apreciables.

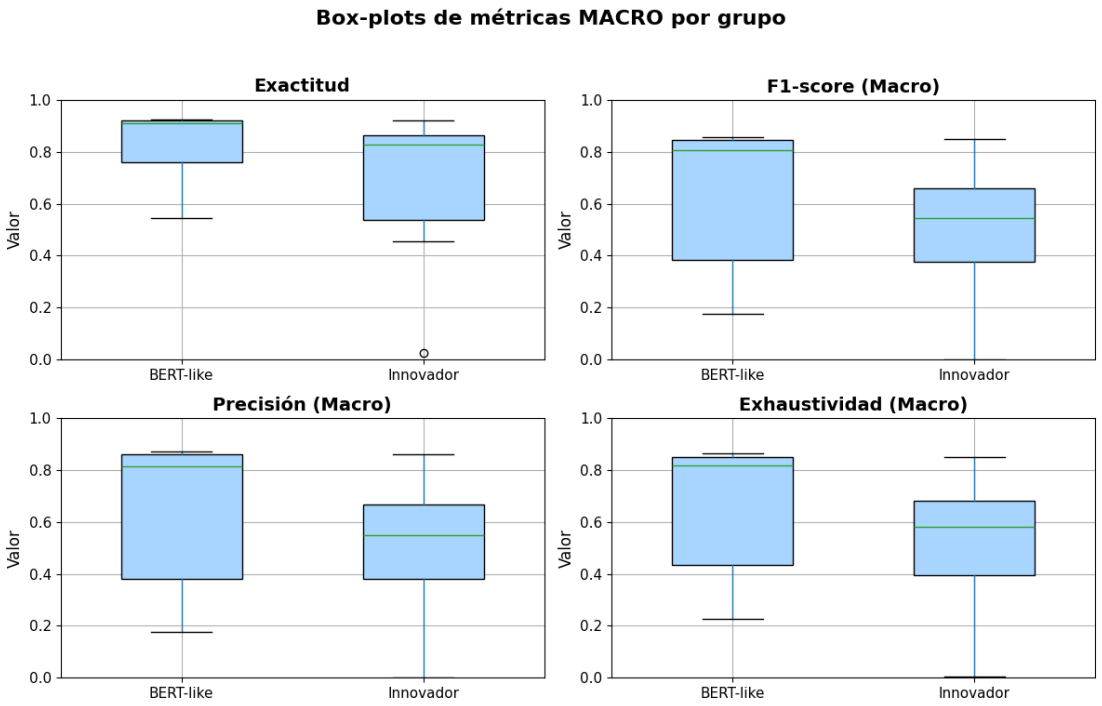

Figura 14: Distribución de métricas *macro* y **exactitud** por tipo de arquitectura.

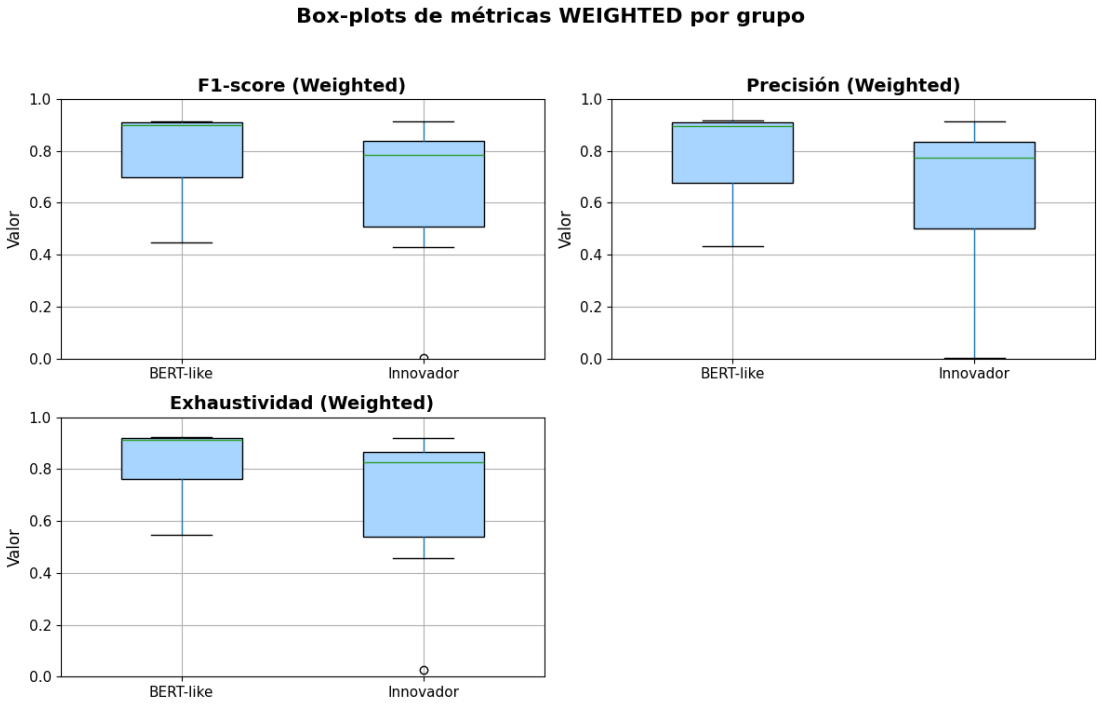

Figura 15: Distribución de métricas *weighted* por tipo de arquitectura.

4.4. Relación entre tamaño del modelo y rendimiento predictivo

Esta sección examina el compromiso entre la complejidad de los modelos —medida en número de parámetros— y su rendimiento en tareas de clasificación multiclase. Se comparan tanto los modelos ajustados mediante *fine-tuning* como los evaluados en modo *zero-shot*, analizando su eficiencia en términos de *F1-score* y su capacidad para generalizar según el tamaño y tipo de arquitectura. A través de visualizaciones comparativas, se identifican patrones de escalabilidad, límites de rendimiento y estrategias estructurales efectivas.

Rendimiento y eficiencia de modelos entrenados mediante ajuste fino

La Figura 16 muestra la relación entre el número de parámetros y el rendimiento en **F1-score** *macro* y *weighted*. Se identifican tres patrones claros: (1) modelos optimizados entre 20–70M de parámetros —como `distilbeto`, `mobilebert` o `distilbert`— logran excelentes resultados con bajo coste; (2) modelos *base* entre 100–150M alcanzan un rendimiento alto pero con retorno marginal decreciente; y (3) modelos muy grandes y multilingües, como `xlm-roberta`, sufren una fuerte degradación por desalineación de dominio e idioma.

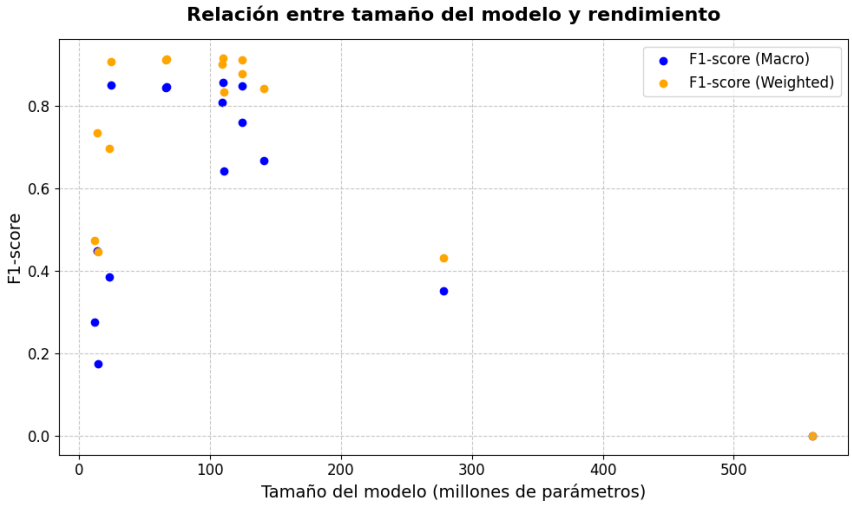

Figura 16: Relación entre número de parámetros y rendimiento tras ajuste fino.

En conjunto, estos resultados permiten extraer varias conclusiones clave:

- **El tamaño óptimo no es el máximo**, sino el que equilibra capacidad y adecuación al dominio de la tarea.
- **La distilación es efectiva**: los modelos compactos bien diseñados igualan o superan a sus equivalentes *base*.
- **El preentrenamiento especializado pesa más que la escala**: un modelo como `beto`, entrenado solo en español, supera con holgura a alternativas multilingües de mayor tamaño.

Rendimiento y eficiencia en escenarios *zero-shot*

Para comprender el equilibrio entre eficiencia computacional y rendimiento predictivo, se ha elaborado la Figura 17, donde se representan dos métricas clave —el **F1-score** *macro* y el **F1-score** *weighted*— en función del número de parámetros de cada modelo (expresado en miles de millones).

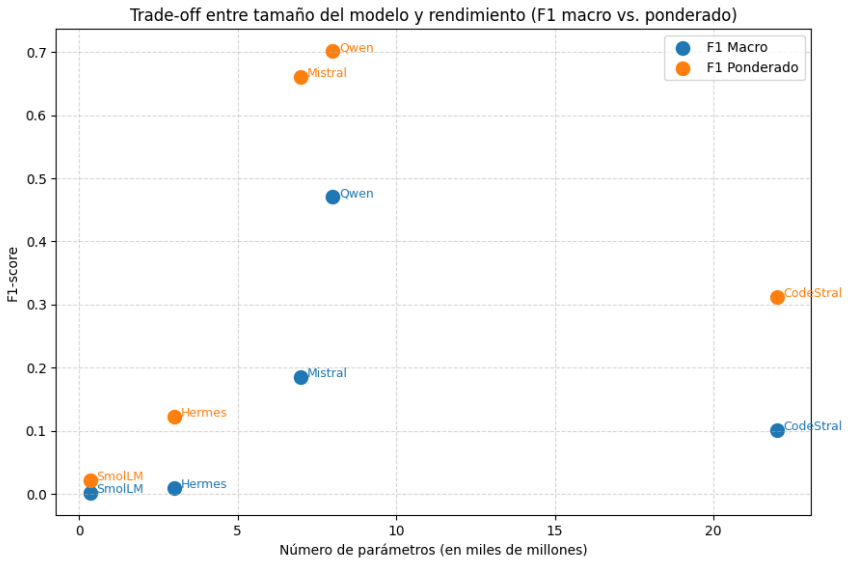

Figura 17: Trade-off entre tamaño del modelo y rendimiento (**F1-score** *macro* vs. *weighted*).

Los resultados reflejan un compromiso evidente entre tamaño, arquitectura y rendimiento en tareas *zero-shot*:

- `Mistral` se posiciona como el modelo más eficiente en relación calidad-tamaño. Con tan solo 7B de parámetros, alcanza uno de los mayores **F1-score** *weighted* y un **F1-score** *macro* competitivo, lo que evidencia una excelente cobertura tanto en clases frecuentes como infrecuentes sin necesidad de una gran escala.

- `Qwen`, con 7B de parámetros, también muestra un rendimiento elevado, siendo el líder en **F1-score** *macro* y ligeramente por encima de `Mistral` en **F1-score** *weighted*, aunque con una diferencia mínima. Ambos ejemplifican un buen equilibrio entre capacidad y coste computacional.

- `CodeStral`, con 22B de parámetros, destaca en rendimiento *weighted*, aunque su **F1-score** *macro* permanece limitado. Su posición revela que su entrenamiento especializado en código le permite sobresalir en clases frecuentes, pero sin generalizar del todo a clases minoritarias.

- `Hermes`, a pesar de tener un tamaño medio (3B), presenta un bajo rendimiento tanto en **F1-score** *macro* como *weighted*. Esto indica que el *fine-tuning* conversacional no optimiza su desempeño para tareas estructuradas de clasificación multiclase sin adaptación previa.

- `SmolLM`, con solo 0.4B de parámetros, registra los peores resultados en ambas métricas. Su tamaño extremadamente reducido no le permite capturar la complejidad semántica necesaria en contextos multiclase sin supervisión.

En resumen, este análisis confirma que un mayor número de parámetros no garantiza un mejor rendimiento. La eficiencia estructural, el tipo de preentrenamiento —particularmente aquel orientado a la comprensión general del lenguaje y la transferencia semántica— y la capacidad de generalización tienen un peso decisivo en el éxito de los modelos en tareas de clasificación multiclase en escenarios *zero-shot*.

4.5. Análisis de errores tras ajuste fino: patrones, frecuencia y causas

Esta sección analiza los errores de clasificación cometidos por los modelos entrenados mediante ajuste fino, con el objetivo de identificar patrones recurrentes, evaluar el impacto de la frecuencia de aparición de etiquetas y productos en su clasificación, y detectar posibles ambigüedades semánticas entre clases.

Etiquetas con mayor incidencia de errores de clasificación

Las Tablas 4.5–4.8 muestran, para cada iteración de la validación cruzada, las diez etiquetas del campo `DESC_PROD` con mayor número de errores de predicción acumulados entre todos los modelos evaluados.

Se incluye también el número de veces que cada etiqueta apareció durante el entrenamiento, lo que permite analizar si su frecuencia influye en la dificultad de clasificación. Estas tablas permiten detectar patrones de confusión frecuentes y destacar etiquetas especialmente problemáticas.

El valor de *errores acumulados* hace referencia a la suma total de errores cometidos por todos los modelos con respecto a una etiqueta concreta durante una misma iteración.

Por ejemplo, en la primera fila de la Tabla 4.5, la etiqueta *Agua mineral sin gas, botella grande_WKB* aparece 147 veces en el conjunto de entrenamiento, pero fue clasificada incorrectamente en 575 ocasiones entre todos los modelos. Esto sugiere una alta confusión con clases similares como *botella muy grande_WKB* o *agua mineral sin gas_WKB*.

Cuadro 4.5: Iteración 1 — 10 etiquetas con más errores entre todos los modelos

Etiqueta (`DESC_PROD`)	**Errores acumulados de clasificación**	**Cantidad de apariciones en el conjunto de entrenamiento**
Agua mineral sin gas, botella grande_WKB	575	147
Agua mineral sin gas, botella muy grande_WKB	430	88
Multipack de latas de cerveza nacional_WKB	399	83
Lata de cerveza nacional_WKB	398	141
Jamón York de pierna cocido y ahumado vendido suelto_NE	377	99
Potito de carne_WKB	311	360
Potito de fruta o verduras_WKB	294	409
Lata de Coca Cola_SB	279	56
Multipack de cerveza nacional_WKB	261	160
Agua mineral sin gas_WKB	243	203

Cuadro 4.6: Iteración 2 — 10 etiquetas con más errores entre todos los modelos

Etiqueta (DESC_PROD)	**Errores acumulados de clasificación**	**Cantidad de apariciones en el conjunto de entrenamiento**
Agua mineral sin gas, botella grande_WKB	499	147
Lata de cerveza nacional_WKB	464	141
Agua mineral sin gas, botella muy grande_WKB	444	88
Multipack de latas de cerveza nacional_WKB	380	83
Jamón York de pierna cocido y ahumado vendido suelto_NE	364	99
Infusión de flores, hierbas o frutas_WKB	297	525
Agua mineral sin gas_WKB	285	203
Lata de Coca Cola_SB	278	56
Potito de fruta o verduras_WKB	230	409
Pan de molde blanco grande_WKB	225	48

Cuadro 4.7: Iteración 3 — 10 etiquetas con más errores entre todos los modelos

Etiqueta (DESC_PROD)	**Errores acumulados de clasificación**	**Cantidad de apariciones en el conjunto de entrenamiento**
Agua mineral sin gas, botella grande_WKB	476	147
Potito de carne_WKB	471	360
Lata de cerveza nacional_WKB	455	141
Agua mineral sin gas_WKB	451	203
Agua mineral sin gas, botella muy grande_WKB	430	89
Multipack de latas de cerveza nacional_WKB	405	83
Jamón York de pierna cocido y ahumado vendido suelto_NE	391	99
Multipack de cerveza nacional_WKB	358	161
Concentrado de zumo_WKB	355	302
Jamón serrano vendido suelto_NE	354	445

Cuadro 4.8: Iteración 4 — 10 etiquetas con más errores entre todos los modelos

Etiqueta (DESC_PROD)	**Errores acumulados de clasificación**	**Cantidad de apariciones en el conjunto de entrenamiento**
Potito de carne_WKB	530	360
Agua mineral sin gas, botella grande_WKB	508	147
Agua mineral sin gas, botella muy grande_WKB	430	89
Jamón York de pierna cocido y ahumado vendido suelto_NE	413	99
Lata de cerveza nacional_WKB	401	141
Multipack de latas de cerveza nacional_WKB	394	84
Aceite de oliva Virgen Extra_WKB	363	531
Agua mineral sin gas_WKB	342	204
Helado 500–1000 ml_WKB	341	398
Potito de fruta o verduras_WKB	338	410

Del análisis de los resultados obtenidos se desprenden dos observaciones preliminares que permiten comprender mejor el patrón de errores en la tarea de clasificación:

1. **Dificultades con productos muy similares.** Se ha observado que ciertas etiquetas con descripciones casi idénticas —como *Agua mineral sin gas, botella grande_WKB*, *Agua mineral sin gas, botella muy grande_WKB* y *Agua mineral sin gas_WKB*— tienden a confundirse con frecuencia. Este comportamiento sugiere que los modelos presentan limitaciones a la hora de captar matices semánticos sutiles, especialmente cuando las diferencias entre clases se reducen a pequeñas variaciones en el formato, el tamaño o la presentación del producto.

2. **Persistencia de errores a lo largo de distintas iteraciones.** Algunas etiquetas aparecen sistemáticamente entre las más conflictivas en varias particiones del conjunto de datos. Esta recurrencia indica que los errores no son aleatorios ni exclusivos de una partición concreta, sino que responden a una tendencia persistente que podría deberse a ambigüedad entre clases, solapamiento semántico o una representación insuficiente de dichas etiquetas durante el entrenamiento.

Estas observaciones motivan un análisis más exhaustivo de los productos específicos que generaron un mayor número de errores de clasificación. En las tablas siguientes se presentan las diez etiquetas con mayor número de errores acumulados en cada iteración del proceso de validación cruzada.

Productos con mayor incidencia de errores de clasificación

Para detectar patrones recurrentes de error, se analizó la clasificación de los productos con mayor número de fallos (`DESC_ART`) en cada *fold*. Este análisis permite identificar artículos con ambigüedad sistemática, ya sea por descripciones poco distintivas, alta similitud con otros productos o presencia de múltiples variantes con etiquetas diferentes.

Las Tablas 4.9–4.12 recogen, por iteración, los 10 productos con más errores acumulados entre todos los modelos. La primera columna muestra su nombre (según el campo `DESC_ART`); la segunda, el total de errores cometidos por los modelos durante la evaluación; y la tercera, la cantidad de veces que la etiqueta correspondiente apareció en el entrenamiento.

Por ejemplo, en la Tabla 4.9, el producto *Agua mineral Font Vella* fue clasificado incorrectamente en 53 ocasiones, pese a haber sido visto 11 veces durante el entrenamiento. Esta alta tasa de error sugiere confusión con productos similares, lo que dificulta su clasificación incluso con cierta representación previa.

Cuadro 4.9: Iteración 1 — 10 productos (DESC_ART) con más errores entre todos los modelos

Producto (DESC_ART)	**Errores acumulados de clasificación**	**Frecuencia en el conjunto de entrenamiento**
Agua mineral Font Vella	53	11
Agua mineral natural (mineralización débil) Fuente Selecta	44	2
Agua mineral Teleno	37	2
Agua mineral Aquarel Nestlé	32	8
Agua mineral natural sin gas baja mineralización	31	1
Agua mineral natural FONT AGUDES	31	2
Cerveza heineken Heineken	30	1
Barbuzano agua mineral sin gas La Palma Barbuzano	30	2
Tomate agata al peso	30	3
Agua mineral solan de cabras Solán de Cabras	29	1

Cuadro 4.10: Iteración 2 — 10 productos (DESC_ART) con más errores entre todos los modelos

Producto (DESC_ART)	**Errores acumulados de clasificación**	**Frecuencia en el conjunto de entrenamiento**
Agua mineral natural Aguadoy	45	2
Agua mineral Font Vella	38	12
Cerveza especial Ambar	31	1
Cerveza sin filtrar El Águila	31	2
Agua mineral natural sin gas Fuente bruma	31	1
Agua potable preparada Los alpes	30	2
Cerveza rubia nacional Pilsen Dorada	30	1
Refresco de cola coca cola Coca-Cola	30	2
Cerveza rubia Steinburg	30	2
Agua mineral Solán de Cabras	30	6

Cuadro 4.11: Iteración 3 — 10 productos (DESC_ART) con más errores entre todos los modelos

Producto (DESC_ART)	**Errores acumulados de clasificación**	**Frecuencia en el conjunto de entrenamiento**
Refresco de cola Coca-Cola	78	14
Agua mineral natural NEVAL	43	2
Agua mineral Font Vella	40	12
Cerveza Heineken	38	9
Agua mineral Aquabona	35	5
Agua mineral natural CORTES	33	3
Agua mineral (mineralización muy débil) Bronchales	31	1
BB Agua mineral BB	30	3
Cerveza Cruzcampo	29	4
Cerveza rubia especial Dorada	28	1

Cuadro 4.12: Iteración 4 — 10 productos (DESC_ART) con más errores entre todos los modelos

Producto (DESC_ART)	**Errores acumulados de clasificación**	**Frecuencia en el conjunto de entrenamiento**
Cerveza Heineken	49	9
Agua mineral natural Font Vella	45	2
Agua mineral Font d'Or	38	2
Agua mineral Bezoya Bezoya	35	1
Refresco de cola zero sin cafeína Coca-Cola	32	3
Agua mineral Solán de Cabras	32	5
Agua mineral Aquarel Nestlé	31	7
Ketchup Heinz	30	11
Manzana roja	30	2
Agua mineral (mineralización débil) Fuente Selecta	30	4

Se aprecia que muchos de los productos con más errores pertenecen a categorías con múltiples variantes muy similares entre sí, como aguas minerales, cervezas o refrescos. Este patrón sugiere que una parte sustancial de los errores se debe a la dificultad para distinguir entre productos con descripciones comerciales próximas, que difieren únicamente en aspectos como la marca, el grado de mineralización o el formato del envase.

Por ejemplo, aparecen recurrentemente productos como *Agua mineral Font Vella*, *Agua mineral Solán de Cabras*, *Agua mineral Aquarel Nestlé* o *Refresco de cola Coca-Cola*, lo que refuerza la hipótesis de que la cercanía semántica entre productos complica la discriminación fina que se espera del modelo.

No obstante, al incorporar la información sobre el número de veces que cada producto aparece durante el entrenamiento, se observa una situación más compleja: algunos de los productos con más errores también se encuentran entre los que tienen menor representación en los datos de entrenamiento, lo cual puede explicar en parte sus malas predicciones. Sin embargo, otros artículos —como *Refresco de cola Coca-Cola* o *Agua mineral Font Vella*— mantienen tasas de error elevadas a pesar de contar con una frecuencia de aparición considerable, lo que sugiere que los errores no se deben únicamente a la escasez de ejemplos, sino también a la confusión con otras clases cercanas.

Este análisis pone de manifiesto la necesidad de estrategias complementarias en la clasificación, como la incorporación de representaciones semánticas más robustas o la redefinición del conjunto de clases mediante agrupaciones más coherentes. Además, subraya que muchas predicciones "erróneas" desde el punto de vista estricto de la etiqueta pueden resultar comprensibles desde una perspectiva semántica, especialmente en dominios como el comercial donde los límites entre clases tienden a ser difusos.

Relación entre la frecuencia de aparición de etiquetas y la tasa de error

Con el objetivo de analizar si existe una relación entre la frecuencia de aparición de las etiquetas en los conjuntos de entrenamiento y la tasa de error cometida por los modelos, se calcularon los coeficientes de correlación de Pearson y Spearman para cada modelo en los cuatro folds. Estos coeficientes permiten cuantificar la relación lineal (Pearson) y monótona (Spearman) entre ambas variables.

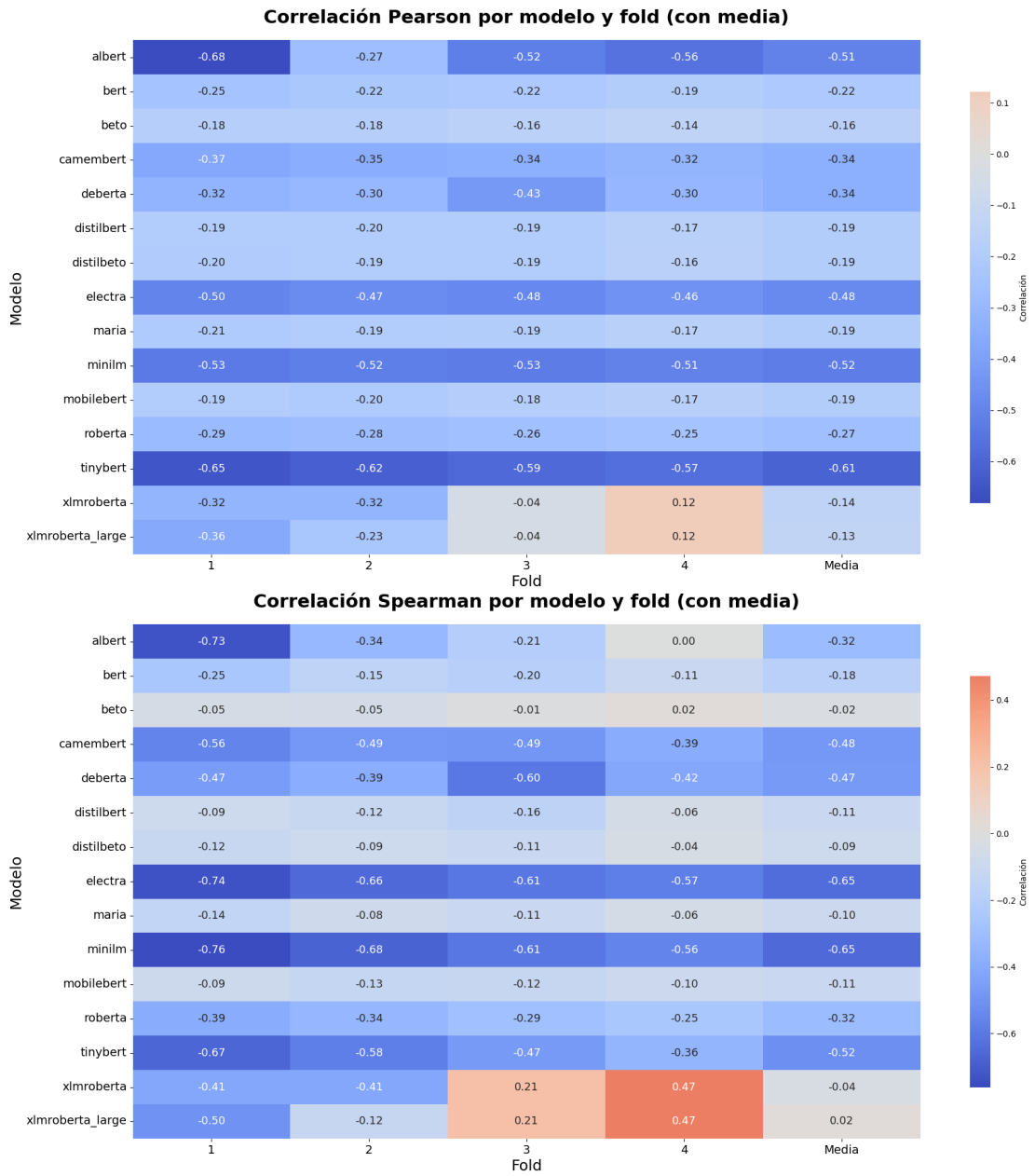

Figura 18: Mapas de calor de correlaciones entre la frecuencia de las etiquetas en el conjunto de entrenamiento y la tasa de error por modelo y fold. Se incluye la media de correlación por modelo.

En la Figura 18 se presentan los mapas de calor resultantes, donde cada fila corresponde a un modelo y cada columna a un fold, incluyendo además una columna adicional que representa la media de correlación entre folds. Los valores negativos indican que, en general, cuanto mayor es la frecuencia de una etiqueta en el conjunto de entrenamiento, menor es su tasa de error, lo cual es consistente con la hipótesis de que una mayor exposición durante el entrenamiento mejora la capacidad del modelo para clasificar correctamente dichas etiquetas.

Se observa que modelos como `tinybert`, `minilm` y `electra` muestran correlaciones particularmente fuertes y negativas en todos los folds, lo cual refuerza esta hipótesis. Por el contrario, modelos como `xlmroberta` y `xlmroberta_large` presentan en algunos casos correlaciones cercanas a cero o incluso positivas, lo que podría indicar una mayor dificultad para beneficiarse de la frecuencia de aparición o un comportamiento más errático respecto a la distribución de etiquetas. Esta falta de correlación clara puede deberse, al menos en parte, al bajo rendimiento general que presentan

ambos modelos en esta tarea, lo cual podría estar enmascarando cualquier relación significativa entre frecuencia y errores.

Análisis de errores de clasificación en escenarios *zero-shot*: etiquetas ambiguas, productos conflictivos y limitaciones del modelo

En esta sección se analizan los errores más representativos cometidos por los modelos evaluados bajo el escenario *zero-shot*. El análisis se centra en identificar las etiquetas y productos más problemáticos, explorar patrones recurrentes de fallo, y examinar posibles causas como la ambigüedad léxica, la proximidad semántica entre clases, o la escasa información contextual.

Etiquetas con mayor incidencia de errores de clasificación

Se presentan a continuación las diez etiquetas del campo `DESC_PROD` con mayor tasa de error bajo el escenario *zero-shot*, para cada uno de los modelos evaluados. La tasa se calcula como el porcentaje de predicciones incorrectas sobre el total de veces que la etiqueta aparece en el conjunto de prueba.

Cada tabla incluye tres columnas principales: la primera indica la etiqueta evaluada; la segunda muestra el número total de errores de clasificación cometidos por el modelo en relación con esa etiqueta; y la tercera recoge cuántos productos del conjunto de validación estaban realmente asociados a dicha etiqueta. La tasa de error se calcula como el cociente entre el número de errores y la cantidad total de productos con esa etiqueta, multiplicado por 100.

Por ejemplo, en la primera fila de la Tabla 4.13, el modelo `codestral` se encontró con productos cuya etiqueta real era `Helado tipo magnum vendido suelto_WKB` en un total de 137 ocasiones durante la evaluación. Sin embargo, en los 137 casos falló al clasificar correctamente dichos productos, lo que equivale a una tasa de error del 100 %.

Cuadro 4.13: Etiquetas con mayor número de errores — `codestral`

Etiqueta (`DESC_PROD`)	**Errores de clasificación**	**Cantidad de instancias asociadas a esta etiqueta**	**Tasa de error (%)**
Helado tipo magnum vendido suelto_WKB	137	137	100.00
Jamón York de pierna cocido y ahumado vendido suelto_NE	131	131	100.00
Ice Tea_WKB	125	125	100.00
Agua mineral sin gas, botella muy grande_WKB	118	118	100.00
Huevos de gallina, en jaula, tamaño medio_NE	61	61	100.00
Infusión de flores, hierbas o frutas_WKB	552	560	98.57
Huevos de gallina, en jaula, tamaño grande_NE	67	68	98.53
Helado 500–1000 ml_WKB	518	529	97.92
Agua mineral sin gas, botella grande_WKB	189	196	96.43
Helado_SB	223	244	91.39

Cuadro 4.14: Etiquetas con mayor número de errores — `hermes`

Etiqueta (`DESC_PROD`)	**Errores de clasificación**	**Cantidad de instancias asociadas a esta etiqueta**	**Tasa de error (%)**
Pasta sin huevo_WKB	433	433	100.00
Agua mineral sin gas, botella grande_WKB	196	196	100.00
Helado tipo cornetto multipack_WKB	181	181	100.00
Vodka_WKB	166	166	100.00
Café tostado mezcla_WKB	165	165	100.00
Leche semidesnatada UHT_WKB	163	163	100.00
Helado tamaño grande_BL	161	161	100.00
Leche entera UHT_WKB	157	157	100.00
Yogurt de frutas_WKB	154	154	100.00
Galletas dulces_WKB	145	145	100.00

Cuadro 4.15: Etiquetas con mayor número de errores — `mistral`

Etiqueta (`DESC_PROD`)	**Errores de clasificación**	**Cantidad de instancias asociadas a esta etiqueta**	**Tasa de error (%)**
Manzana Golden Delicious o Granny Smith_NE	65	65	100.00
Jamón prensado vendido suelto_NE	51	51	100.00
Arroz grano largo, no precocido_WKB	43	43	100.00
Leche evaporada_WKB	37	37	100.00
Leche fresca semidesnatada_WKB	37	37	100.00
Mermelada de albaricoque o fresa_BL	37	37	100.00
Cerveza multipack_SB	35	35	100.00
Filete de caballa en lata en salsa de tomate o aceite vegetal_WKB	35	35	100.00
Cornflakes_SB	34	34	100.00
Copos de maíz_WKB	30	30	100.00

Cuadro 4.16: Etiquetas con mayor número de errores — `qwen`

Etiqueta (`DESC_PROD`)	**Errores de clasificación**	**Cantidad de instancias asociadas a esta etiqueta**	**Tasa de error (%)**
Jamón prensado vendido suelto_NE	51	51	100.00
Arroz grano largo, no precocido_WKB	43	43	100.00
Ketchup_SB	40	40	100.00
Pasta sin huevo_BL	36	36	100.00
Filete de caballa en lata en salsa de tomate o aceite vegetal_WKB	35	35	100.00
Helado tamaño grande_BL	160	161	99.38
Yogurt de frutas_WKB	153	154	99.35
Cacao en polvo_WKB	84	85	98.82
Pan blanco barra pequeña_NE	78	80	97.50
Pasta sin huevo_WKB	422	433	97.46

Cuadro 4.17: Etiquetas con mayor número de errores — `smollm`

Etiqueta (`DESC_PROD`)	**Errores de clasificación**	**Cantidad de instancias asociadas a esta etiqueta**	**Tasa de error (%)**
Infusión de flores, hierbas o frutas_WKB	358	358	100.00
Aceite de oliva Virgen Extra_WKB	281	281	100.00
Jamón serrano vendido suelto_NE	255	255	100.00
Yogurt con sabor a frutas_WKB	238	238	100.00
Concentrado de zumo_WKB	158	158	100.00
Helado tipo cornetto multipack_WKB	155	155	100.00
Queso en lonchas_WKB	136	136	100.00
Potito de fruta o verduras_WKB	132	132	100.00
Vodka_WKB	126	126	100.00
Helado_SB	116	116	100.00

Se observa que varias etiquetas conflictivas aparecen de forma reiterada en los errores de distintos modelos. Esta recurrencia puede atribuirse a diversos factores. Por un lado, muchas de estas etiquetas presentan descripciones largas o compuestas, lo que incrementa la ambigüedad y dificulta su interpretación precisa. Por otro, suelen corresponder a productos con múltiples variantes muy similares —como helados, yogures o aguas minerales—, lo que favorece errores por sustitución o invención. Además, el uso de sufijos como `_WKB`, `_NE` o `_SB`, carentes de significado semántico para los modelos, puede añadir ruido y obstaculizar el proceso de clasificación.

Se identifica también una tendencia clara: los productos pertenecientes a categorías afines —por ejemplo, diferentes tipos de agua, cortes de carne o formatos de yogur— tienden a confundirse entre sí. Esto refuerza la hipótesis de que la similitud semántica entre clases es una fuente relevante de error, y que los modelos tienen dificultades para captar con precisión las diferencias sutiles entre categorías próximas.

Asimismo, los errores aumentan cuando las descripciones de entrada son breves, ambiguas o carecen de atributos distintivos. Por el contrario, cuando se incluye información contextual adicional, como categorías explícitas o marcas reconocibles, los modelos tienden a ofrecer respuestas más acertadas. Esto sugiere que la calidad y riqueza del contexto léxico tiene un efecto directo sobre el rendimiento, especialmente en entornos *zero-shot* donde no se ha aplicado ajuste fino previo.

Productos con mayor incidencia de errores de clasificación

En las Tablas 4.18–4.22 se recogen los productos concretos que registraron un mayor número de errores de clasificación por parte de los modelos evaluados en modo *zero-shot*. Cada tabla muestra, para cada producto, el número de errores cometidos al clasificarlo, el total de veces que ese producto aparece en los datos, y la tasa de error resultante (porcentaje de fallos respecto al total de apariciones).

Por ejemplo, en la primera fila de la Tabla 4.18, el producto `Agua mineral Font Vella` aparece 16 veces en el conjunto de evaluación. El modelo `codestral` falló en las 16 ocasiones al clasificarlo, lo que se traduce en una tasa de error del 100 %. Este comportamiento indica que el modelo no fue capaz de identificar correctamente este producto en ningún caso.

Cuadro 4.18: Productos con mayor tasa de error — `codestral`

Producto (`DESC_ART`)	**Errores de clasificación**	**Número de instancias**	**Tasa de error (%)**
Agua mineral Font Vella	16	16	100.00
Agua mineral Solán de Cabras	8	8	100.00
Agua mineral Sierra de Cazorla	6	6	100.00
Agua mineral Teleno	6	6	100.00
Agua mineral natural (mineralización débil) Fuente Selecta	6	6	100.00
Agua mineral Aquabona	7	8	87.50
Aceite de oliva virgen extra Carbonell	6	7	85.71
Agua mineral natural Font Vella	6	7	85.71
Agua mineral Aquarel Nestlé	10	12	83.33
Aceite de oliva virgen extra Hojiblanca	8	13	61.54

Cuadro 4.19: Productos con mayor tasa de error — `hermes`

Producto (`DESC_ART`)	**Errores de clasificación**	**Número de instancias**	**Tasa de error (%)**
Ketchup Heinz	16	16	100.00
Leche semidesnatada Central Lechera Asturiana	14	14	100.00
Cerveza Heineken	13	13	100.00
Leche entera Central Lechera Asturiana	13	13	100.00
Refresco de cola Pepsi	13	13	100.00
Mandarina	11	11	100.00
Ketchup Orlando	9	9	100.00
Aceite de oliva virgen extra Hojiblanca	10	13	76.92
Agua mineral Font Vella	13	16	81.25
Refresco de cola Coca-Cola	12	20	60.00

Cuadro 4.20: Productos con mayor tasa de error — `mistral`

Producto (`DESC_ART`)	**Errores de clasificación**	**Número de instancias**	**Tasa de error (%)**
Cerveza Heineken	13	13	100.00
Leche entera Central Lechera Asturiana	13	13	100.00
Ketchup original Prima	9	9	100.00
Café soluble natural Nescafé	8	8	100.00
Queso de untar natural Philadelphia	7	7	100.00
Arroz largo La Cigala	6	6	100.00
Café soluble descafeinado Nescafé	6	6	100.00
Cerveza rubia Heineken	6	6	100.00
Agua mineral Solán de Cabras	7	8	87.50
Agua mineral Font Vella	13	16	81.25

Cuadro 4.21: Productos con mayor tasa de error — `qwen`

Producto (`DESC_ART`)	**Errores de clasificación**	**Número de instancias**	**Tasa de error (%)**
Ketchup Heinz	16	16	100.00
Leche entera Central Lechera Asturiana	13	13	100.00
Queso de untar natural Philadelphia	7	7	100.00
Refresco de cola light Coca-Cola	7	7	100.00
Refresco de cola light Coca-Cola Light	7	7	100.00
Agua mineral Solán de Cabras	7	8	87.50
Aceite de oliva suave 0,4° Carbonell	6	7	85.71
Agua mineral Font Vella	12	16	75.00
Cerveza Heineken	7	13	53.85
Refresco de cola Coca-Cola	8	20	40.00

Cuadro 4.22: Productos con mayor tasa de error — `smollm`

Producto (`DESC_ART`)	**Errores de clasificación**	**Número de instancias**	**Tasa de error (%)**
Refresco de cola Pepsi	13	13	100.00
Mayonesa Hellmann's	8	8	100.00
Queso de untar natural Philadelphia	7	7	100.00
Aceite de oliva virgen extra Valdezarza	5	5	100.00
Agua mineral natural Aguadoy	5	5	100.00
Chocolatina Kinder Delice Kinder	5	5	100.00
Cola normal Coca-Cola	5	5	100.00
Aceite de oliva suave 0,4° Carbonell	5	7	71.43
Refresco de cola light Coca-Cola	5	7	71.43
Refresco de cola Coca-Cola	12	20	60.00

El hecho de que ciertos productos acumulen errores en múltiples modelos sugiere que no se trata de fallos aleatorios, sino de un patrón sistemático vinculado a la proximidad semántica entre clases. En particular, los errores tienden a concentrarse en categorías conceptualmente relacionadas, como aguas minerales, aceites, cervezas, refrescos o productos lácteos. Esta recurrencia indica que la similitud entre estos elementos induce a los modelos a sobregeneralizar durante la asignación de etiquetas.

Este comportamiento es consistente con los resultados observados previamente en modelos ajustados mediante *fine-tuning*, lo que refuerza la hipótesis de que los modelos de lenguaje a gran escala, incluso sin reentrenamiento específico, tienden a agrupar bajo una misma representación semántica productos con estructuras léxicas o funciones similares. Como consecuencia, etiquetas distintas —pero léxicamente próximas o funcionalmente equivalentes, como distintas marcas de agua embotellada— tienden a ser tratadas como intercambiables, especialmente cuando la entrada no proporciona información contextual suficiente para diferenciarlas.

Este fenómeno plantea un desafío relevante en escenarios donde la distinción entre variantes es crítica, y subraya la necesidad de incorporar mecanismos adicionales de desambiguación. Entre ellos destacan estrategias como la ingeniería de *prompts*, la inclusión de metadatos adicionales o el reentrenamiento parcial del modelo para mejorar su capacidad de discriminación semántica fina.

Relación entre frecuencia de aparición y tasa de error relativa

En esta sección se examina la relación entre la frecuencia de aparición de cada etiqueta y su tasa de error relativa, entendida como el cociente entre el número de errores cometidos por el modelo y el número total de apariciones de dicha etiqueta. Esta métrica resulta especialmente útil en escenarios *zero-shot*, donde los modelos no han sido afinados explícitamente para las clases objetivo, y se evalúa su capacidad de generalización frente a etiquetas de diversa frecuencia.

Estudiar esta relación permite identificar si los modelos tienden a generalizar mejor en etiquetas comunes —más similares a los patrones presentes en los datos de preentrenamiento—, mientras que podrían fallar en etiquetas menos frecuentes o semánticamente más específicas, para las cuales no disponen de representaciones tan refinadas. Por ejemplo, un modelo puede clasificar correctamente productos etiquetados como *"Aceite de oliva Virgen Extra WKB"*, una categoría habitual con numerosos ejemplos, pero fallar en clases más especializadas como *"Botella de vino tinto regional europeo, Cabernet Sauvignon_WKB"*, donde el nivel de especificidad técnica es mayor y la cantidad de ejemplos en los datos de evaluación es notablemente inferior.

La Figura 19 muestra los coeficientes de correlación de Pearson y Spearman entre la frecuencia de aparición y la tasa de error relativa para cada modelo evaluado. En general, los valores son bajos, lo que sugiere que no existe una relación fuerte entre ambas variables. No obstante, se observan algunas diferencias relevantes:

- `CodeStral` presenta la correlación más alta (Pearson = 0,19; Spearman = 0,05), lo que sugiere una ligera tendencia a cometer más errores relativos en etiquetas frecuentes, posiblemente debido a su mayor diversidad semántica.
- `SmolLM` muestra una correlación negativa de tipo no lineal (Spearman = −0,21), indicando una mayor tasa de error en etiquetas poco frecuentes o especializadas.
- `Qwen`, `Hermes` y `Mistral` muestran correlaciones cercanas a cero, lo que sugiere que sus errores no se concentran ni en etiquetas muy frecuentes ni en las más raras, sino que se distribuyen de forma más uniforme.

Estos resultados proporcionan información complementaria al rendimiento global de los modelos, revelando posibles debilidades frente a clases con escasa representación. Esta limitación puede resultar especialmente crítica en aplicaciones donde la cobertura semántica debe ser amplia y robusta frente a etiquetas minoritarias.

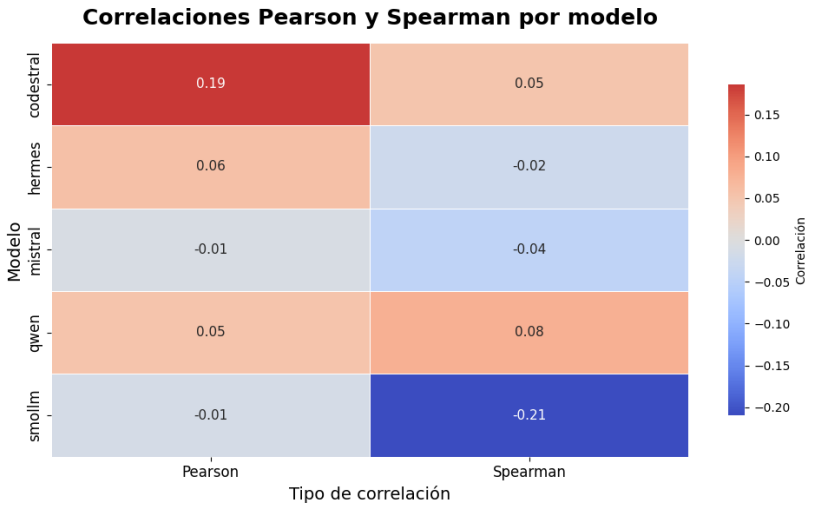

Figura 19: Correlación entre la frecuencia de aparición de las etiquetas y la tasa de error relativa, por modelo.

4.6. Discusión de los resultados obtenidos

En esta sección se analizan críticamente los principales hallazgos derivados de la evaluación de los modelos aplicados a la tarea de clasificación de productos. Se examina el rendimiento alcanzado tanto por los modelos ajustados específicamente mediante técnicas de *fine-tuning*, como por aquellos utilizados en escenarios de inferencia directa sin ejemplos previos (*zero-shot*). La discusión contempla no solo los resultados cuantitativos, sino también las limitaciones metodológicas y las implicaciones prácticas, ofreciendo una visión global sobre la idoneidad de cada enfoque en función del contexto de aplicación.

Evaluación crítica de los modelos ajustados mediante ajuste fino (fine-tuning)

Los resultados obtenidos confirman que los modelos de lenguaje preentrenados, una vez adaptados mediante técnicas de *fine-tuning*, pueden ser aplicados con éxito a tareas de clasificación multiclase de texto en dominios específicos como el comercio minorista. Modelos como `beto`, `distilbeto` y `maria` alcanzaron niveles de exactitud y F1-score ponderado superiores al 90 %, lo que los convierte en candidatos viables para ser integrados en sistemas reales de categorización automática de productos.

Además, el buen rendimiento de modelos compactos como `mobilebert` o `distilbeto` sugiere que es posible implementar soluciones eficientes en entornos con restricciones de memoria o capacidad computacional, como aplicaciones móviles o sistemas embebidos.

Por tanto, la aplicabilidad práctica de estos modelos es alta, especialmente en entornos donde se dispone de datos etiquetados y se requiere una clasificación precisa y automática a partir de descripciones textuales.

A pesar del buen rendimiento global observado tras el ajuste fino de los modelos, es importante tener en cuenta varias limitaciones metodológicas que condicionan la generalización de los resultados:

- **Homogeneidad en la estrategia de ajuste:** Todos los modelos se entrenaron bajo un mismo esquema de hiperparámetros (épocas, tasa de aprendizaje, tamaño de lote, etc.), sin adaptación específica a sus características arquitectónicas. Esto pudo limitar el potencial de

modelos que requerían ajustes más finos o estrategias personalizadas para converger de forma óptima.

- **Falta de búsqueda de hiperparámetros:** No se llevó a cabo una búsqueda sistemática de hiperparámetros (como *grid search* o *random search*) debido a restricciones computacionales. Esto reduce la posibilidad de haber alcanzado el rendimiento óptimo de cada modelo, especialmente en configuraciones sensibles como el número de épocas o el tamaño de lote.

- **Sesgo inducido por el desbalance de clases:** Aunque se usaron métricas ponderadas para evaluar el rendimiento, el entrenamiento no incluyó técnicas activas de reequilibrio (como *class weighting* o *oversampling*), lo que pudo favorecer las clases frecuentes y perjudicar la clasificación de etiquetas minoritarias.

- **Etiquetas artificiales y específicas:** Muchas clases están definidas por nombres largos, con sufijos como `_WKB` o `_SB`, que no tienen significado semántico y pueden confundir al modelo. Además, varias etiquetas son muy similares entre sí, lo que hace que el modelo requiera una alta sensibilidad semántica para distinguirlas correctamente.

- **Entrenamiento limitado por capacidad computacional:** La elección de entrenar durante solo 4 épocas y con configuraciones modestas fue una decisión pragmática, condicionada por la disponibilidad de recursos locales sin acceso a GPUs de alto rendimiento. Esto limita el alcance de las conclusiones, especialmente para modelos más grandes o con mayores necesidades de ajuste.

Estas limitaciones ayudan a contextualizar tanto los aciertos como los errores de los modelos. En particular, explican por qué algunas arquitecturas no alcanzaron su máximo rendimiento esperado, por qué ciertas etiquetas se clasifican peor pese a su frecuencia, y por qué persisten errores entre productos conceptualmente próximos. Futuras líneas de trabajo podrían centrarse en adaptar los hiperparámetros por modelo, introducir técnicas de reequilibrio, y rediseñar las etiquetas o los campos de entrada para facilitar el aprendizaje semántico.

Análisis del comportamiento de los modelos en escenarios *zero-shot*

Los resultados obtenidos sugieren que `Qwen` y `Mistral` son los modelos más prometedores para escenarios de clasificación multiclase en contexto *zero-shot*, combinando un rendimiento competitivo con una eficiencia razonable. En particular:

- `Qwen` destaca como el modelo con mejor **F1-score** *macro* y alta **precisión** general, siendo especialmente competente para cubrir tanto clases frecuentes como infrecuentes.

- `Mistral`, con un tamaño más reducido (7B), logra un rendimiento muy cercano, lo que lo convierte en una opción viable para entornos con recursos limitados.

Desde una perspectiva práctica, modelos como `Qwen` y `Mistral` ofrecen un buen equilibrio entre **rendimiento**, **coste computacional** y **capacidad de generalización**, lo que los hace adecuados para aplicaciones reales sin requerir ajuste fino adicional.

A pesar de la utilidad de este análisis, los resultados deben interpretarse considerando ciertas limitaciones metodológicas y debilidades observadas en el comportamiento de los modelos:

- **Ausencia de contexto adicional:** Al tratarse de una tarea *zero-shot*, los modelos no disponían de ejemplos etiquetados, definiciones ni instrucciones específicas sobre el dominio. Esto afectó especialmente a productos con descripciones ambiguas o genéricas (*e.g.*, *Helado_SB*, *Vodka_WKB*), donde los modelos fallaron por falta de señales contextuales que ayudaran a distinguir entre clases similares.

- **Convenciones de etiquetado artificiales:** Los sufijos como `_WKB`, `_NE` o `_SB`, al no poseer significado semántico explícito, no fueron comprendidos por los modelos, provocando errores en etiquetas formalmente distintas pero conceptualmente cercanas. Esto fue evidente en confusiones sistemáticas en productos como *huevos*, *helados* o *jamón*.

- **Desbalance en la distribución de clases:** Las etiquetas más frecuentes dominaron el rendimiento *weighted*, mientras que las minoritarias sufrieron mayor tasa de error. Esto afectó directamente al **F1-score** *macro*, especialmente en modelos como `CodeStral` o `Hermes`, que mostraron mayor dependencia del sesgo hacia clases frecuentes.

- **Dificultad para diferenciar productos similares:** Muchos errores ocurrieron entre categorías con diferencias sutiles (*agua mineral con/sin gas*, *leche entera/semidesnatada*, *tipos de pasta*). Esta fragilidad se relaciona con la falta de entrenamiento explícito en taxonomías finas de productos y refuerza la necesidad de un preprocesamiento semántico más riguroso o de *prompts* más guiados.

- **Respuestas no estructuradas y entrada simplificada:** Las predicciones se generaron en formato libre, sin plantillas ni delimitadores, lo que pudo llevar a respuestas inconsistentes o desviadas, especialmente en modelos de menor tamaño como `SmolLM`, que mostraron confusión incluso en clases frecuentes.

Estas limitaciones explican en parte por qué los modelos fallan más en etiquetas largas, ambiguas, con variantes próximas o sufijos artificiales. También justifican la diferencia entre métricas *macro* y *weighted*, y sugieren vías de mejora: uso de etiquetas reescritas con lenguaje natural, *prompts* instructivos, o soporte mediante ejemplos de pocos disparos (*few-shot*) en futuras iteraciones experimentales.

Comparación entre modelos entrenados mediante *fine-tuning* y los validados por *zero-shot*

Se comparan los modelos entrenados específicamente para la tarea (*fine-tuned*) con aquellos evaluados sin ajuste previo (*zero-shot*).

Los modelos *fine-tuned* muestran un rendimiento claramente superior. Arquitecturas como `beto`, `distilbeto` o `maria` superan el 90 % de **exactitud**, con valores de **F1-score** *weighted* (ponderados) por encima de 0,91 y métricas *macro* mayores a 0,85, lo que evidencia una buena cobertura incluso en clases minoritarias.

En contraste, los modelos *zero-shot* presentan resultados dispares: la **exactitud** varía entre el 1,77 % de `SmolLM` y el 70,23 % de `Qwen`, y los valores de *F1-score macro* rara vez superan 0,5. Modelos como `Hermes`, `CodeStral` o `SmolLM` incurren en errores sistemáticos, con tasas superiores al 90 % en múltiples etiquetas.

Aunque `Qwen` o `Mistral` captan en parte el sentido de productos frecuentes, esta capacidad no compensa su imprecisión categórica. La ambigüedad léxica, la cercanía semántica entre clases y la falta de contexto afectan notablemente su rendimiento.

Los errores tienden a concentrarse en categorías similares —como aguas, helados o lácteos— y se agravan con sufijos técnicos como `_WKB`, `_NE` o `_SB`, que introducen ruido en la predicción.

En cambio, los modelos *fine-tuned* muestran mayor adaptación semántica y generalización, incluso ante clases poco representadas. Siempre que haya datos anotados, el ajuste fino sigue siendo la estrategia más eficaz y fiable.

Capítulo 5

Discusión y Conclusiones

5.1. Introducción

Este capítulo resume y analiza los resultados obtenidos al aplicar dos enfoques de clasificación con modelos de lenguaje de gran tamaño (LLMs): *fine-tuning* y *zero-shot learning*. La comparación se basa en su rendimiento sobre un conjunto de datos desbalanceado de productos de supermercado, teniendo en cuenta también las limitaciones computacionales, las métricas de evaluación y las particularidades de los modelos empleados.

5.2. Discusión de resultados

Los resultados permiten valorar tanto el comportamiento de los LLMs en tareas de clasificación comercial como la viabilidad de su uso en entornos con recursos limitados.

Los modelos entrenados con *fine-tuning* mostraron una adaptación sólida al dominio, alcanzando buenos niveles de precisión incluso bajo configuraciones restringidas. Modelos como `distilbert`, `electra`, `minilm` o `deberta` ofrecieron un equilibrio entre eficiencia y rendimiento, con validación cruzada a cuatro particiones para asegurar robustez y minimizar sesgos.

Estas pruebas se realizaron en un entorno local, sin GPUs ni servidores de alto rendimiento. Esta limitación condicionó la selección de modelos, el número de épocas (reducido a cuatro) y la elección de técnicas eficientes como `CrossEntropyLoss` y el optimizador AdamW. Aun así, los resultados confirman que es posible entrenar modelos eficaces con medios modestos si se aplica una metodología rigurosa.

El análisis de errores mostró dificultades en etiquetas poco representadas o semánticamente próximas (por ejemplo, tipos de agua o cerveza), y una clara relación entre la frecuencia de las clases y la tasa de error.

En cuanto al enfoque *zero-shot*, los modelos tuvieron un desempeño más limitado, especialmente en etiquetas complejas o ambiguas, debido a la falta de adaptación al dominio. Sin embargo, gracias al uso de versiones optimizadas en formato `GGUF` mediante LM Studio, fue posible evaluarlos en CPU sin infraestructura avanzada.

Lejos de ser un obstáculo, la restricción computacional refuerza el valor práctico del estudio. Este demuestra que es viable entrenar y comparar modelos competitivos con recursos accesibles, promoviendo el uso de LLMs en entornos reales como pequeñas empresas, proyectos educativos o desarrollos individuales.

5.3. Conclusiones generales

Del análisis realizado se extraen las siguientes conclusiones:

- Los LLMs preentrenados pueden adaptarse con eficacia a tareas de clasificación mediante *fine-tuning*, incluso con recursos limitados.
- El rendimiento depende en gran medida de la frecuencia y claridad semántica de las etiquetas; las menos frecuentes o ambiguas presentan mayores tasas de error.
- El enfoque *zero-shot*, aunque atractivo por su simplicidad, requiere prompts más elaborados o esquemas *few-shot* para acercarse al rendimiento del *fine-tuning*.
- La validación cruzada ha sido clave para garantizar comparaciones justas y evaluar la variabilidad entre particiones.
- No se observaron ventajas sistemáticas entre modelos en inglés frente a los entrenados en español, aunque estos últimos resultaron más eficientes para textos exclusivamente en este idioma.
- Es posible entrenar y evaluar modelos competitivos en entornos locales, lo que abre oportunidades para su aplicación práctica más allá de laboratorios especializados.

5.4. Líneas futuras de investigación

Este trabajo abre diversas posibilidades de continuación:

- Aplicar el enfoque a otros dominios comerciales con taxonomías distintas o textos más técnicos.
- Explorar el *few-shot learning* mediante ejemplos contextuales en los *prompts*.
- Integrar preprocesamiento semántico o enriquecimiento textual para mejorar la discriminación entre productos similares.
- Emplear técnicas como *class weighting* o *focal loss* para equilibrar el rendimiento en clases minoritarias.
- Evaluar arquitecturas más recientes o diseñadas específicamente para clasificación jerárquica o multietiqueta.
- Investigar métodos de entrenamiento y despliegue eficientes en entornos con recursos limitados, fomentando el uso práctico de estos modelos sin depender de infraestructura avanzada.

En resumen, los LLMs actuales, combinando adecuadamente ajuste fino y evaluación realista, pueden ser herramientas eficaces para la clasificación automática de productos. No obstante, su despliegue industrial requiere soluciones más adaptativas, eficientes e interpretables que garanticen coherencia semántica y sostenibilidad computacional.

Capítulo 6

Discussion and Conclusions

6.1. Introduction

This chapter summarizes and analyzes the results obtained by applying two classification approaches using Large Language Models (LLMs): *fine-tuning* and *zero-shot learning*. The comparison focuses on their performance over an imbalanced dataset of supermarket products, also considering computational constraints, evaluation metrics, and the specific characteristics of the models used.

6.2. Discussion of Results

The results allow us to assess both the behavior of LLMs in product classification tasks and the feasibility of their use in resource-constrained environments.

Models trained via *fine-tuning* showed strong adaptation to the domain, achieving solid accuracy even under limited configurations. Models such as `distilbert`, `electra`, `minilm`, and `deberta` struck a good balance between efficiency and performance, with four-fold cross-validation ensuring robustness and reduced bias.

All experiments were conducted locally, without access to GPUs or high-performance servers. This constraint influenced the choice of models, the number of epochs (reduced to four), and the use of efficient techniques such as `CrossEntropyLoss` and the AdamW optimizer. Even so, the results confirm that it is possible to train effective models with modest resources when a rigorous methodology is applied.

Error analysis revealed difficulties with underrepresented or semantically similar labels (e.g., types of water or beer), and a clear inverse correlation between label frequency and error rate.

Regarding the *zero-shot* approach, models performed less effectively, especially with complex or ambiguous labels, due to the lack of domain-specific adaptation. However, the use of optimized `GGUF` versions deployed via LM Studio enabled CPU-based evaluation without advanced infrastructure.

Far from being a limitation, the computational constraint adds practical value to this study. It demonstrates that competitive models can be trained and compared with accessible resources, promoting the use of LLMs in real-world settings such as small businesses, educational projects, or individual development environments.

6.3. General Conclusions

The following conclusions can be drawn from the analysis:

- Pretrained LLMs can be effectively adapted to product classification tasks via *fine-tuning*, even with limited computational resources.

- Model performance is strongly influenced by label frequency and semantic clarity; rare or ambiguous labels tend to produce higher error rates.
- The *zero-shot* approach, while appealing for its simplicity, requires more elaborate prompts or *few-shot* configurations to approach the performance of *fine-tuned* models.
- Cross-validation was essential to ensure fair comparisons and evaluate behavior across different data splits.
- No systematic advantage was observed between English general-purpose models and those adapted to Spanish, although monolingual Spanish models proved more efficient for tasks involving only Spanish text.
- It is feasible to train and evaluate competitive models in local environments, enabling practical applications beyond specialized labs.

6.4. Future Research Directions

This work opens several avenues for future research:

- Apply the approach to other commercial domains with different taxonomies or more technical product descriptions.
- Explore *few-shot learning* by incorporating contextual examples into prompts.
- Integrate semantic preprocessing or text enrichment techniques to better distinguish between similar products.
- Apply techniques such as *class weighting* or *focal loss* to balance performance across minority classes.
- Evaluate newer or more specialized architectures designed for hierarchical or multi-label classification.
- Investigate efficient training and deployment strategies for resource-limited environments, supporting the practical use of LLMs without relying on advanced infrastructure.

In conclusion, current LLMs, when properly combined with fine-tuning and realistic evaluation strategies, can serve as effective tools for automated product classification. However, their deployment in industrial contexts requires more adaptive, efficient, and interpretable solutions that ensure semantic consistency and computational sustainability.

Bibliografía

[1] J. Alammar and M. Grootendorst, *Hands-On Large Language Models: Language Understanding and Generation*, 2024.

[2] *Large Language Models: A Deep Dive*, 2024. Documento interno facilitado por el profesor.

[3] *Understanding Large Language Models*, 2024. Documento interno facilitado por el profesor.

[4] Hugging Face. *The AI community building the future*. Disponible en: https://huggingface.co. [Accedido: julio de 2025].

[5] LM Studio. *Run large language models locally*. Disponible en: https://lmstudio.ai. [Accedido: julio de 2025].

Apéndice A

Apéndice

Materiales adicionales, scripts y recursos complementarios pueden consultarse en el siguiente enlace:

`Google Drive - Material Complementario`